Hawassa University
Institute of Technology
Faculty of Informatics
Department of Computer Science

Advanced Programming (CoSc3052)

Chapter 3: Java GUI using JavaFX - Comprehensive Course Note

Compiled by Elias Jarso, ***for CS 3rd Year Sec-3***

October, 2025GC

Course Context & Historical Overview: The Evolution of Java GUI

Welcome to the exciting world of Java Graphical User Interface (GUI) programming! Before we dive deep into JavaFX, it's crucial to understand the landscape from which it emerged. Java has a rich history of GUI toolkits, each attempting to solve problems and offer better ways to build interactive applications.

What are AWT, Swing, and JavaFX?

1. **AWT (Abstract Window Toolkit):**
   - **What it is:** AWT was Java's original GUI toolkit, introduced with Java 1.0. It provided a basic set of components like buttons, text fields, and windows.
   - **Why it emerged:** To allow Java applications to have a graphical interface that could run on different operating systems (write once, run anywhere - WORA).
   - **How Java moved over it:** AWT components were "heavyweight." This means they relied heavily on the underlying operating system's native GUI components. When you created an AWT button, Java internally mapped it to a Windows button, a macOS button, or a Linux button. While achieving WORA, this led to a "lowest common denominator" look and feel, and inconsistencies across platforms. Developers often struggled with platform-specific bugs and limited customization.
     - In the context of Java AWT, the **"lowest common denominator"** refers to the **most basic set of features and capabilities that are guaranteed to be available on *every* supported platform** (Windows, macOS, Linux/Motif, etc.).
     - The Core Idea: AWT's original philosophy was: "**If a UI component (like a button or text field) doesn't exist natively on one platform, we cannot provide it in AWT.**" This meant that the AWT API could only include components and features that were present in the native toolkits of *all* the operating systems it aimed to support.
     - IN AWT 1.0, **The "Lowest Common Denominator" was:**
       `Button`, `TextField`, `Checkbox`, `List`, `Label`, `Scrollbar`, etc.
     - These components were available everywhere, so AWT could provide them via a common Java API, which then called the native OS functions to render them. This is why AWT is called a **"heavyweight"** component set—every AWT component was a direct wrapper around a native, platform-specific peer.
     - What Was *Excluded* by this Approach? Consider more advanced components that were *not* universally available:
       - **Tables:** Windows had a ListView, macOS might not have had a standard table, Motif was basic. Therefore, **AWT had no `Table` component.**
       - **Tree Views:** Windows had a TreeView, but it wasn't a standard component elsewhere. Therefore, **AWT had no `Tree` component.**
       - **Rich Text Editors:** Capabilities varied wildly. Therefore, AWT's `TextArea` was very basic.
       - **Tooltips:** Not a standard feature on all platforms. Excluded.
       - **Tabbed Panes:** Not universally available. Excluded.

     Any feature that was not present on even one major platform had to be left out of the core AWT API to maintain true WORA. This severely limited the richness and sophistication of the UIs that could be built with pure AWT.

     The Consequences of the "Lowest Common Denominator" Approach

     1. **"Look and Feel":** Your application would look like a Windows app on Windows, a Mac app on a Mac, and a Motif app on Unix. This was the original goal. However, the *behavior* could also differ slightly (e.g., how a text field handles focus), leading to subtle, hard-to-find bugs.
     2. **Limited Customization:** Since the components were native, you couldn't easily change their appearance. You were stuck with the default look of the underlying OS. Want a blue button with rounded corners on Windows 95? Nearly impossible.
     3. **Platform-Specific Bugs:** Because AWT was a thin layer over native code, bugs in the native OS's UI toolkit would surface in your Java application. Debugging meant figuring out if the bug was in your Java code, the AWT implementation, or the OS itself.
     4. **Inconsistencies:** The "write once, run anywhere" promise was sometimes broken by these platform-specific bugs and behavioral differences. You might have to write code to handle specific platform quirks, which defeated the purpose of WORA.

2. **Swing:**

- **What it is:** Swing was introduced as part of the Java Foundation Classes (JFC) with Java 1.2. It was designed to replace AWT and offer a more robust and flexible GUI solution.
- **Why it emerged:** To overcome AWT's limitations, particularly its heavyweight nature and inconsistent look and feel. Swing components are "lightweight," meaning they are drawn entirely by Java code, not by native OS components (except for the top-level window).
- **How Java moved over it:** Swing provided a richer set of components, better customization capabilities (pluggable look and feel), and improved performance compared to AWT. It became the de facto standard for Java desktop applications for over a decade. However, Swing, while powerful, could be complex to work with, especially for rich graphics and media. Its rendering model was not optimized for modern GPU acceleration.

The Contrast: How Swing Solved "Lowest Common Denominator"

Swing, which came later, abandoned the "lowest common denominator" model. Its philosophy was:

"We will implement our own UI components in pure Java, *emulating* the look and feel. We don't rely on native components at all."

This is why Swing is **"lightweight"**—the components are drawn by Java code onto a blank canvas. This allowed Swing to offer a rich set of components (tables, trees, tabbed panes, etc.) that behaved *identically* on every platform.

- **Advantage:** Rich features, consistent behavior, highly customizable.
- **Disadvantage:** Could feel "foreign" or slow on some platforms because it didn't use the native OS's UI widgets.

3. **JavaFX:**

- **What it is:** JavaFX is the successor to Swing for developing rich internet applications and desktop applications. It was initially released in 2008 and later integrated into JDK 7.
- **Why it emerged:** To address the limitations of Swing in the modern era of rich media, animations, touch interfaces, and GPU-accelerated graphics. Swing's architecture wasn't designed for these requirements. JavaFX aimed to provide a modern, high-performance, and developer-friendly platform for building visually stunning applications.
- **How Java moved over it:** Oracle actively promoted JavaFX as the future of client-side Java. It offered a declarative way to build UIs (FXML), powerful styling with CSS, a robust animation framework, and direct support for 2D/3D graphics and media. While Swing applications still exist and are maintained, new development in Java GUI is strongly encouraged to use JavaFX.

Key Features of JavaFX that Differentiate It:

- **Modern Architecture:** Built from the ground up to leverage modern hardware, especially GPUs, for smooth rendering and animations.
- **Scene Graph:** A hierarchical tree of nodes that represents the visual structure of the UI, making it intuitive to manipulate and manage UI elements.
- **FXML:** A declarative XML-based language for defining UI layouts, separating UI design from application logic. This allows designers and developers to work more efficiently.
- **CSS Styling:** Enables styling the UI using standard Cascading Style Sheets, offering powerful customization and consistent look and feel.
- **Properties and Binding:** A powerful mechanism for creating reactive UIs where changes in data automatically update the UI, reducing boilerplate code and improving maintainability.
- **Rich Media Support:** Integrated support for audio, video, and web content (WebView).
- **Animations and Effects:** A comprehensive animation API for creating visually engaging transitions and effects.
- **Cross-Platform:** Still adheres to the WORA principle, running on Windows, macOS, and Linux.

Chapter 3: Java GUI using JavaFX

This chapter will guide you through building modern graphical applications with JavaFX. We will adopt a hands-on, step-by-step approach, ensuring you understand not just *what* to do, but *why* and *how* it works. For every concept, we will explore all available implementation methods and provide extensive, in-depth demo examples. We will build our understanding progressively, focusing on code-side concepts first before introducing FXML.

3.1. JavaFX Architecture and Program Structure

Understanding the underlying architecture is key to mastering JavaFX. It's designed for high performance and developer productivity.

JavaFX Architecture: The Scene Graph and Rendering Pipeline

The core of JavaFX's visual structure is the **Scene Graph**.

- **Scene Graph:** Think of the Scene Graph as a tree-like data structure where each node represents a visual element in your application.
    - **Nodes:** The fundamental building blocks of the Scene Graph. Everything visible in a JavaFX application is a Node.
        - **Branch Nodes (Parent Nodes):** Nodes that can contain other nodes (e.g., Layout Panes like VBox, HBox, BorderPane). They manage the layout and position of their children.
        - **Leaf Nodes:** Nodes that cannot contain other nodes (e.g., Button, Label, Rectangle, ImageView). These are the actual UI controls or shapes.
    - **Hierarchy:** Nodes are arranged hierarchically. A Scene contains the root node, which in turn can contain other nodes, forming a tree. This hierarchy defines rendering order and event propagation.
- **Rendering Pipeline:** When the Scene Graph is modified (e.g., a button is clicked, text changes, an animation runs), JavaFX's rendering pipeline kicks in.
    - It efficiently determines what needs to be redrawn.
    - It leverages modern GPUs (Graphics Processing Units) for hardware acceleration, resulting in smooth animations and high-performance graphics.
    - This is a significant advantage over older toolkits like Swing, which primarily relied on the CPU for rendering.

Detailed Breakdown of a Basic JavaFX Program Structure

Every JavaFX application extends the javafx.application.Application class. This class provides the entry point and manages the application's lifecycle.

Application Lifecycle:

1. **main(String[] args):** The standard Java entry point. In a JavaFX application, its primary role is to call Application.launch(args);. This method is responsible for setting up the JavaFX environment.
2. **init():** This method is called after the application is launched but *before* the start() method. It's an optional method you can override to perform initialization tasks that don't require a visible UI, such as loading resources, setting up databases, or creating background services. It runs on the main thread, but *not* the JavaFX Application Thread.
3. **start(Stage primaryStage):** This is the **most important** method you must override. It's called when the application is ready to display its UI. This is where you construct your Scene Graph, create your Scene, and set it on the Stage. It runs on the **JavaFX Application Thread**.
4. **stop():** This method is called when the application is shutting down, after the last window has closed. It's an optional method used to perform cleanup tasks, such as saving user preferences, closing database connections, or releasing resources. It runs on the JavaFX Application Thread.

Clear Explanation of Stage, Scene, and their Relationship

Think of a theatrical play:

- **Stage:** The main window of your application. It's the top-level container provided by the operating system. You can have multiple Stage objects in an application (e.g., a main window and a separate "About" dialog). The start() method receives the primary Stage object.
    - Properties: Title, icon, resizable, full screen, width, height, etc.
- **Scene:** The content *inside* the Stage. It's the container for your entire Scene Graph. A Stage can hold only one Scene at a time. You can switch between different Scene objects on the same Stage to change the application's view.
    - Properties: Width, height, fill color, stylesheets. Crucially, it holds the **root node** of your Scene Graph.
- **Relationship:** The Stage is the window frame, and the Scene is the canvas placed within that frame. You first build your UI (nodes) into a root node, then put that root node into a Scene, and finally set that Scene onto the Stage.

`

`

Simple "Hello JavaFX" Program

```
// File: HelloJavaFX.java
import javafx.application.Application; // The base class for JavaFX applications
import javafx.scene.Scene;           // The container for all visual content
import javafx.scene.control.Label;   // A simple UI control for displaying text
import javafx.scene.layout.StackPane; // A layout container to arrange nodes
import javafx.stage.Stage;           // The top-level window

public class HelloJavaFX extends Application {
    /**
     * The main entry point for all JavaFX applications.
     * The start method is called after the init method has returned,
     * and after the system is ready for the application to begin running.
     *
     * @param primaryStage The primary stage for this application, onto which
     *                     the application scene can be set. The first stage
     *                     will always be provided by the platform.
     */
    @Override
    public void start(Stage primaryStage) {
        // Step 1: Create a UI Control (a Leaf Node)
        // A Label is used to display a short text string.
        Label helloLabel = new Label("Hello, JavaFX!");

        // Step 2: Create a Layout Container (a Branch Node / Parent Node)
        // A StackPane stacks its children on top of each other, centering them by default.
        // It's simple for a single element.
        StackPane root = new StackPane();
        root.getChildren().add(helloLabel); // Add the label to the layout pane

        // Step 3: Create a Scene
        // The Scene holds the root node of our UI and specifies its dimensions.
        // Here, 300 width and 200 height.
        Scene scene = new Scene(root, 300, 200);

        // Step 4: Set the Scene on the Primary Stage
        // The Stage is the main window provided by the OS.
        primaryStage.setTitle("My First JavaFX App"); // Set the window title
        primaryStage.setScene(scene);                 // Place our scene onto the stage

        // Step 5: Show the Stage
        primaryStage.show();                          // Make the window visible
    }

    /**
     * The main method is ignored in JavaFX applications when running as a JAR,
     * but it is useful for launching the application from an IDE.
     * It calls Application.launch() which sets up the JavaFX environment
     * and eventually calls the start() method.
     *
     * @param args Command line arguments
     */
    public static void main(String[] args) {
        launch(args); // This is the standard way to launch a JavaFX application
    }

    // Optional: Override init() for pre-start initialization
    @Override
    public void init() throws Exception {
        System.out.println("Application initializing...");
        super.init(); // Always call super.init()
    }

    // Optional: Override stop() for post-exit cleanup
    @Override
    public void stop() throws Exception {
        System.out.println("Application shutting down...");
        super.stop(); // Always call super.stop()
    }
}
```

Line-by-Line Explanation:

- import javafx.application.Application;: Imports the base class for all JavaFX applications.
- import javafx.scene.Scene;: Imports the class representing the content area of a window.
- import javafx.scene.control.Label;: Imports a UI control to display text.
- import javafx.scene.layout.StackPane;: Imports a layout container.
- import javafx.stage.Stage;: Imports the class representing a top-level window.
- public class HelloJavaFX extends Application {: Our application class must extend Application.
- @Override public void start(Stage primaryStage) {: This is the core method where we build our UI. primaryStage is the main window provided by the system.
- Label helloLabel = new Label("Hello, JavaFX!");: We create an instance of Label and set its text. This is a **leaf node**.
- StackPane root = new StackPane();: We create an instance of StackPane. This is a **branch node** (a layout container) that will hold our Label.
- root.getChildren().add(helloLabel);: We add the helloLabel to the StackPane. getChildren() returns an ObservableList of nodes that the parent manages.
- Scene scene = new Scene(root, 300, 200);: We create a Scene object. Its constructor takes the root node (our StackPane) and the desired width and height for the scene.
- primaryStage.setTitle("My First JavaFX App");: We set the title that appears in the window's title bar.
- primaryStage.setScene(scene);: We attach our scene to the primaryStage.
- primaryStage.show();: This command makes the primaryStage (and thus our Scene and Label) visible on the screen.
- public static void main(String[] args) { launch(args); }: This standard main method is used to launch the JavaFX application. Application.launch() handles the setup and eventually calls our start() method.
- init() and stop(): These are overridden optional methods to show when they are called in the lifecycle. You'd place non-UI initialization and cleanup code here respectively.

To run this code:

1. Save it as HelloJavaFX.java.
2. Compile: javac --module-path <path_to_javafx_sdk>/lib --add-modules javafx.controls,javafx.fxml HelloJavaFX.java (Adjust javafx.controls and javafx.fxml as needed, javafx.controls is sufficient here).
3. Run: java --module-path <path_to_javafx_sdk>/lib --add-modules javafx.controls HelloJavaFX

(Replace <path_to_javafx_sdk> with the actual path to your JavaFX SDK installation.)

3.2. JavaFX Layout Components

Layout panes are crucial for arranging UI controls and other nodes within your application. They automatically manage the position and size of their children, making your UI responsive to window resizing. JavaFX offers a rich set of layout panes, each with its own purpose.

Understanding Layout Concepts:

- **Managed vs. Unmanaged:** Most nodes are "managed" by their parent layout pane. The layout pane decides their size and position. You can set a node as "unmanaged" if you want to manually control its position (using setLayoutX, setLayoutY), but this is generally discouraged for complex UIs as it loses the benefits of automatic layout.
- **Preferred, Min, Max Size:** Nodes have preferred, minimum, and maximum sizes. Layout panes consider these when arranging children.
    - setPrefWidth/Height(): The desired size.
    - setMinWidth/Height(): The smallest size the node can be.
    - setMaxWidth/Height(): The largest size the node can be.

Property	Meaning
`minWidth` / `minHeight`	The **smallest size** the node can be. Layouts will not make it smaller than this.
`prefWidth` / `prefHeight`	The **desired (natural)** size the node *wants* to be. Layout panes usually respect this unless space is limited or excessive.
`maxWidth` / `maxHeight`	The **largest size** the node can grow to. Layouts will not stretch it beyond this.

So, In managed layouts, size hints guide the layout manager. In unmanaged or manual layouts (like Pane), your setPrefSize(w,h) directly sets the visible size.

- **Padding and Spacing:**
  - **Padding:** The empty space between the layout pane's border and its children.
  - **Spacing:** The empty space *between* the children within the layout pane.

Let's explore the core layout panes with examples. …NEXT……

VBox (Vertical Box)

- **Purpose:** Arranges all its children in a single vertical column.
- **Typical Use Cases:** Forms, lists of items, navigation menus.
- **Properties:**
  - spacing: The amount of vertical space between each child.
  - padding: Insets (top, right, bottom, left) around the VBox.
  - alignment: How children are aligned horizontally within the VBox's width (e.g., Pos.TOP_LEFT, Pos.CENTER, Pos.BASELINE_RIGHT).
  - VBox.setVgrow(Node child, Priority value): A static method that hints to the VBox how a child should grow vertically when extra space is available (NEVER, ALWAYS, SOMETIMES).

Demo Example: Basic Form with VBox

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class VBoxDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        Label nameLabel = new Label("Name:");
        TextField nameField = new TextField();
        nameField.setPromptText("Enter your name"); // Placeholder text

        Label emailLabel = new Label("Email:");
        TextField emailField = new TextField();
        emailField.setPromptText("Enter your email");

        Button submitButton = new Button("Submit");
        Button cancelButton = new Button("Cancel");

        // Create a VBox to arrange elements vertically
        VBox root = new VBox(10); // 10 pixels spacing between children
        root.setPadding(new Insets(20)); // 20 pixels padding around the VBox itself
        root.setAlignment(Pos.CENTER_LEFT); // Align children to the center-left

        // Add children in the desired vertical order
        root.getChildren().addAll(nameLabel, nameField, emailLabel, emailField, submitButton,
cancelButton);

        Scene scene = new Scene(root, 350, 300); // Set scene size
        primaryStage.setTitle("VBox Form Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

- In this example, VBox stacks the labels, text fields, and buttons vertically.
- root.setSpacing(10): Adds 10 pixels of vertical space between each node.
- root.setPadding(new Insets(20)): Adds 20 pixels of padding on all sides within the VBox.
- root.setAlignment(Pos.CENTER_LEFT): Centers the entire column of children horizontally within the VBox's available width, and aligns them to the left.

HBox (Horizontal Box)

- **Purpose:** Arranges all its children in a single horizontal row.
- **Typical Use Cases:** Toolbars, button groups, status bars.
- **Properties:**
  - spacing: The amount of horizontal space between each child.
  - padding: Insets around the HBox.
  - alignment: How children are aligned vertically within the HBox's height (e.g., Pos.CENTER_RIGHT, Pos.BOTTOM_CENTER).
  - HBox.setHgrow(Node child, Priority value): A static method that hints how a child should grow horizontally when extra space is available.

Demo Example: Button Bar with HBox

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.HBox;
import javafx.stage.Stage;

public class HBoxDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        Button newButton = new Button("New");
        Button openButton = new Button("Open");
        Button saveButton = new Button("Save");
        Button exitButton = new Button("Exit");

        // Create an HBox to arrange buttons horizontally
        HBox root = new HBox(15); // 15 pixels spacing between buttons
        root.setPadding(new Insets(10)); // 10 pixels padding around the HBox
        root.setAlignment(Pos.CENTER); // Center all buttons horizontally and vertically within the HBox

        // Add buttons
        root.getChildren().addAll(newButton, openButton, saveButton, exitButton);

        Scene scene = new Scene(root, 400, 100);
        primaryStage.setTitle("HBox Button Bar Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

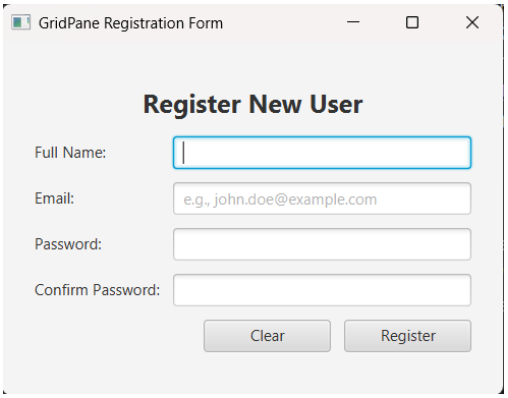
- HBox arranges the buttons horizontally.
- root.setSpacing(15): Adds 15 pixels of horizontal space.
- root.setAlignment(Pos.CENTER): Centers the row of buttons both horizontally and vertically.

GridPane

- **Purpose:** Arranges children in a flexible grid of rows and columns, similar to a table layout.
- **Typical Use Cases:** Complex forms, dashboards, spreadsheets, Tic-Tac-Toe games.
- **Properties:**
  - hgap, vgap: Horizontal and vertical spacing between grid cells.
  - padding: Insets around the GridPane.
  - alignment: How the entire grid is aligned within the GridPane's space.

- GridPane.setColumnIndex(Node child, int index), GridPane.setRowIndex(Node child, int index): Static methods to specify the cell for a child.
- GridPane.setRowSpan(Node child, int value), GridPane.setColumnSpan(Node child, int value): Static methods to make a child span multiple rows/columns.
- GridPane.setHalignment(Node child, HPos value), GridPane.setValignment(Node child, VPos value): Static methods to align a child *within* its cell.
- getColumnConstraints(), getRowConstraints(): Lists of ColumnConstraints and RowConstraints objects to define column widths (e.g., percentage, fixed, Hgrow) and row heights.

Demo Example: Registration Form wit h GridPane (Lab Practice-1 compatible)




```
import javafx.application.Application;
import javafx.geometry.HPos;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.PasswordField;
import javafx.scene.control.TextField;
import javafx.scene.layout.ColumnConstraints;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.Priority;
import javafx.stage.Stage;

public class GridPaneRegistrationForm extends Application {

    @Override
    public void start(Stage primaryStage) {
        // Create the GridPane
        GridPane gridPane = new GridPane();
        gridPane.setAlignment(Pos.CENTER); // Center the entire grid within the scene
        gridPane.setHgap(10); // Horizontal gap between columns
        gridPane.setVgap(10); // Vertical gap between rows
        gridPane.setPadding(new Insets(25, 25, 25, 25)); // Padding around the grid

        // Define Column Constraints for better responsiveness
        // Column 0: Label, fixed width or default
        ColumnConstraints col1 = new ColumnConstraints();
        col1.setHgrow(Priority.NEVER); // Labels usually don't need to grow
        gridPane.getColumnConstraints().add(col1);

        // Column 1: Input Field, grows to fill space
        ColumnConstraints col2 = new ColumnConstraints();
        col2.setHgrow(Priority.ALWAYS); // Text fields should grow horizontally
        gridPane.getColumnConstraints().add(col2);

        // Create UI Controls
        Label titleLabel = new Label("Register New User");
        titleLabel.setStyle("-fx-font-size: 20px; -fx-font-weight: bold;"); // Simple inline
styling

        Label nameLabel = new Label("Full Name:");
        TextField nameField = new TextField();
        nameField.setPromptText("e.g., John Doe");

        Label emailLabel = new Label("Email:");
        TextField emailField = new TextField();
        emailField.setPromptText("e.g., john.doe@example.com");

        Label passwordLabel = new Label("Password:");
        PasswordField passwordField = new PasswordField();

        Label confirmPasswordLabel = new Label("Confirm Password:");
        PasswordField confirmPasswordField = new PasswordField();

        Button registerButton = new Button("Register");
```

```
        registerButton.setPrefWidth(100); // Set preferred width for consistency
        Button clearButton = new Button("Clear");
        clearButton.setPrefWidth(100);

        // Add controls to the grid (Node, columnIndex, rowIndex, columnSpan, rowSpan)
        // Row 0: Title Label, spans 2 columns
        gridPane.add(titleLabel, 0, 0, 2, 1);
        GridPane.setHalignment(titleLabel, HPos.CENTER); // Center the title within its spanned
cells

        // Row 1: Name
        gridPane.add(nameLabel, 0, 1);
        gridPane.add(nameField, 1, 1);

        // Row 2: Email
        gridPane.add(emailLabel, 0, 2);
        gridPane.add(emailField, 1, 2);

        // Row 3: Password
        gridPane.add(passwordLabel, 0, 3);
        gridPane.add(passwordField, 1, 3);

        // Row 4: Confirm Password
        gridPane.add(confirmPasswordLabel, 0, 4);
        gridPane.add(confirmPasswordField, 1, 4);

        // Row 5: Buttons (using an HBox inside a GridPane cell for horizontal alignment)
        HBox buttonBox = new HBox(10); // 10px spacing between buttons
        buttonBox.setAlignment(Pos.CENTER_RIGHT); // Align buttons to the right within the HBox
        buttonBox.getChildren().addAll(clearButton, registerButton);

        gridPane.add(buttonBox, 1, 5); // Add the HBox to cell (1, 5)
        GridPane.setHalignment(buttonBox, HPos.RIGHT); // Align the buttonBox to the right within
its cell

        // Event handlers (simple for now, detailed later)
        registerButton.setOnAction(e -> {
            System.out.println("Register button clicked!");
            System.out.println("Name: " + nameField.getText());
            System.out.println("Email: " + emailField.getText());
            // In a real app, you'd perform validation and registration logic here
        });

        clearButton.setOnAction(e -> {
            nameField.clear();
            emailField.clear();
            passwordField.clear();
            confirmPasswordField.clear();
            System.out.println("Form cleared.");
        });


        Scene scene = new Scene(gridPane, 450, 400);
        primaryStage.setTitle("GridPane Registration Form");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

- This example creates a robust form. Notice how GridPane.setColumnIndex and GridPane.setRowIndex are used to place elements precisely.
- setHgap and setVgap add spacing between cells.
- ColumnConstraints are used to make the input fields (TextField, PasswordField) grow horizontally with the window.

- GridPane.setHalignment(titleLabel, HPos.CENTER) centers the title across its spanned columns.
- An HBox is nested within a GridPane cell to arrange the "Clear" and "Register" buttons horizontally and align them to the right. This demonstrates how layouts can be effectively nested.

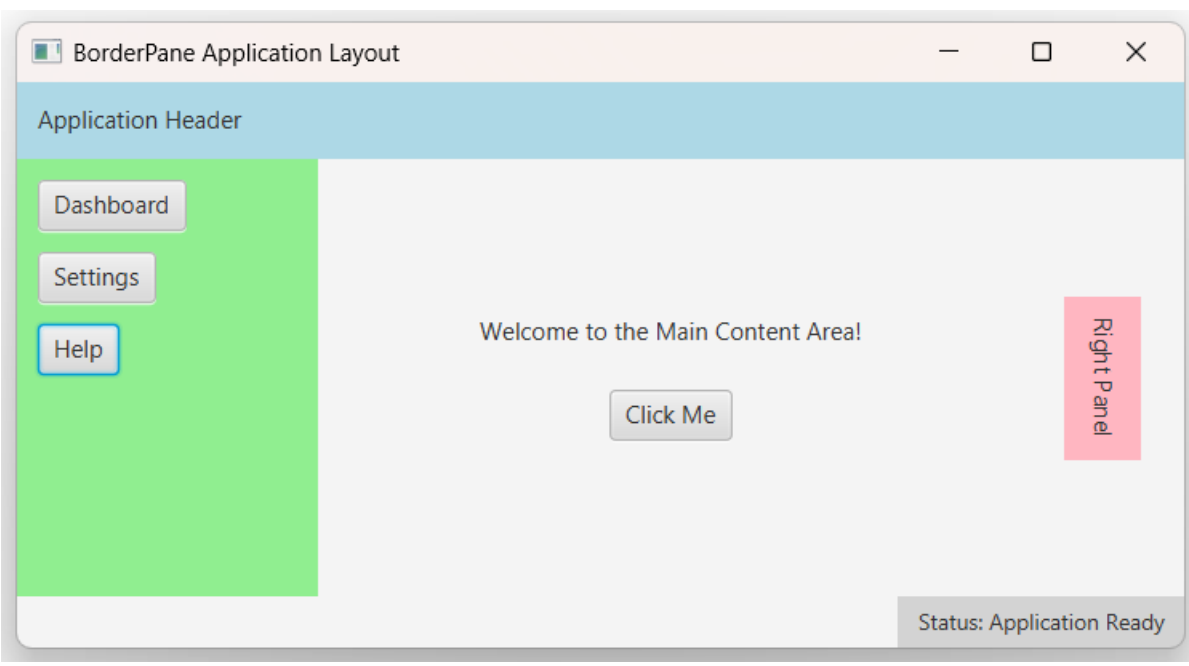
BorderPane

- **Purpose:** Divides the layout into five regions: Top, Bottom, Left, Right, and Center. Each region can hold a single node (which can itself be another layout pane).
- **Typical Use Cases:** Main application layouts (e.g., menu bar at top, status bar at bottom, navigation on left, content in center).
- **Properties:**
  - BorderPane.setTop(Node node), setBottom(Node node), setLeft(Node node), setRight(Node node), setCenter(Node node): Static methods to place nodes in specific regions.
  - BorderPane.setAlignment(Node child, Pos value): Static method to align a child within its designated region (e.g., BorderPane.setAlignment(button, Pos.CENTER)).
  - BorderPane.setMargin(Node child, Insets value): Static method to set margins around a child within its region.

Demo Example: Application Layout with BorderPane

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class BorderPaneDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        BorderPane root = new BorderPane();

        // TOP region: Header (e.g., menu bar, app title)
        HBox topBox = new HBox(10);
        topBox.setPadding(new Insets(10));
        topBox.setAlignment(Pos.CENTER_LEFT);
        topBox.setStyle("-fx-background-color: #ADD8E6;"); // Light blue background
        topBox.getChildren().add(new Label("Application Header"));
        root.setTop(topBox);

        // LEFT region: Navigation (e.g., sidebar)
        VBox leftBox = new VBox(10);
        leftBox.setPadding(new Insets(10));
        leftBox.setPrefWidth(150); // Give it a fixed width
        leftBox.setStyle("-fx-background-color: #90EE90;"); // Light green background
        leftBox.getChildren().addAll(
                new Button("Dashboard"),
                new Button("Settings"),
                new Button("Help")
        );
        root.setLeft(leftBox);

        // CENTER region: Main content area
        VBox centerBox = new VBox(20);
        centerBox.setPadding(new Insets(20));
        centerBox.setAlignment(Pos.CENTER);
        centerBox.getChildren().addAll(
                new Label("Welcome to the Main Content Area!"),
                new Button("Click Me")
        );
```

```
        root.setCenter(centerBox);

        // BOTTOM region: Status bar
        Label statusBar = new Label("Status: Application Ready");
        statusBar.setPadding(new Insets(5, 10, 5, 10)); // Top, Right, Bottom, Left
        statusBar.setStyle("-fx-background-color: #D3D3D3; -fx-font-size: 11px;"); // Light gray
        // An HBox to hold the status bar, aligned to the right, then center the HBox in the bottom
        HBox bottomBox = new HBox();
        bottomBox.setAlignment(Pos.BOTTOM_RIGHT);
        bottomBox.getChildren().add(statusBar);
        root.setBottom(bottomBox);


        // RIGHT region (optional): Could be ads, related info, etc.
        // For this example, we'll leave it empty or add a simple placeholder.
        Label rightLabel = new Label("Right Panel");
        rightLabel.setRotate(90); // Rotate text for effect
        rightLabel.setPadding(new Insets(10));
        rightLabel.setStyle("-fx-background-color: #FFB6C1;"); // Light pink
        StackPane rightWrapper = new StackPane(rightLabel); // Use Stack

        root.setRight(rightWrapper);

        Scene scene = new Scene(root, 700, 500);
        primaryStage.setTitle("BorderPane Application Layout");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }

}
```



- This example demonstrates a typical application layout, nesting `HBox` and `VBox` within `BorderPane` regions.
- Each region (Top, Left, Center, Bottom, Right) can contain **only one direct child**. To place multiple items, you must nest another layout pane (like `HBox` or `VBox`) as that single child.
- root.setTop(topBox)` places the `topBox` (an `HBox`) in the top region.
- Margins can be applied to children within the `BorderPane` using `BorderPane.setMargin()`.

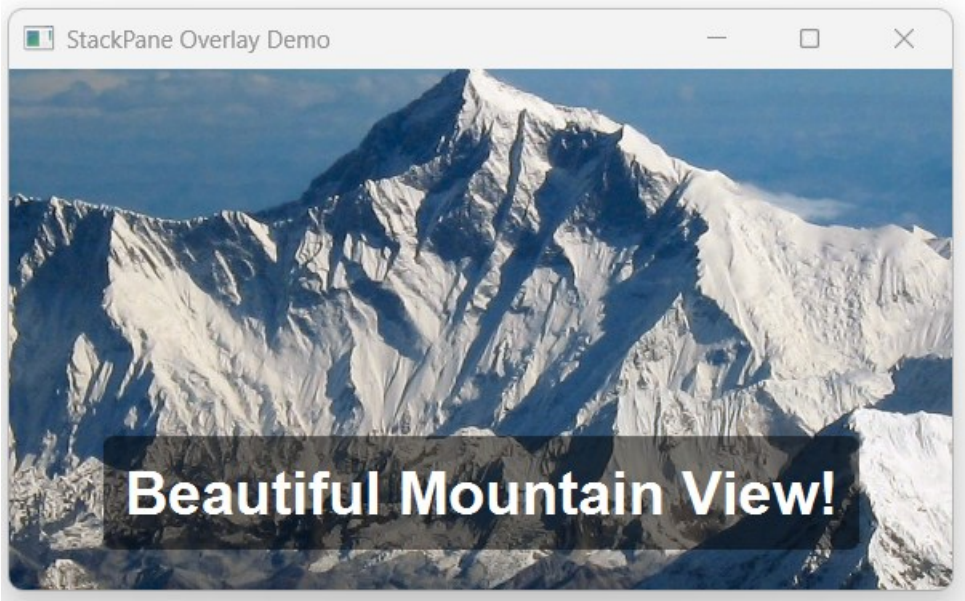
StackPane

- **Purpose**: Places all its children in a single stack, one on top of the other, centered by default.
- **Typical Use Cases:** Overlaying elements (e.g., loading spinners over content, notifications, displaying multiple views where only one is visible at a time).
- **Properties**:
  - `StackPane.setAlignment(Node child, Pos value)`: Static method to align a specific child within the stack pane (e.g., `Pos.TOP_LEFT`, `Pos.BOTTOM_RIGHT`).
  - `StackPane.setMargin(Node child, Insets value)`: Static method to set margins around a specific child.

Demo Example: Image Overlay with StackPane

```
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.StackPane;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;
import javafx.stage.Stage;

public class StackPaneDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        // Image to be in the background
        // Make sure you have an image file (e.g., "forest.jpg") in your project's root or
resources folder
        // You can download a sample image or use a URL if you prefer:
        // Image image = new Image("file:path/to/your/image.jpg");
        // For demonstration, let's try to load from a common path.
        // If this fails, replace with a valid image path or a web URL.
        Image backgroundImage = new
Image(getClass().getResource("/images/background.png").toExternalForm());
//make sure you have save background.png inside your project folder inside 'images' subfolder

        ImageView backgroundImageView = new ImageView(backgroundImage);
        backgroundImageView.setFitWidth(600); // Scale image to fit
        backgroundImageView.setPreserveRatio(true); // Maintain aspect ratio

        // Label to be overlaid on top of the image
        Label overlayText = new Label("Beautiful Mountain View!");
        overlayText.setFont(Font.font("Arial", FontWeight.BOLD, 30));
        overlayText.setTextFill(Color.WHITE); // White text for contrast
        overlayText.setStyle("-fx-background-color: rgba(0, 0, 0, 0.5); -fx-padding: 10px; -fx-
background-radius: 5px;");

        // Create a StackPane and add children. The order matters: last added is on top.
        StackPane root = new StackPane();
        root.getChildren().addAll(backgroundImageView, overlayText);

        // Align the text to the bottom center of the StackPane
        StackPane.setAlignment(overlayText, Pos.BOTTOM_CENTER);
        StackPane.setMargin(overlayText, new Insets(0, 0, 20, 0)); // Margin from bottom

        Scene scene = new Scene(root); // Size will be determined by image view for initial scene
        primaryStage.setTitle("StackPane Overlay Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

- The StackPane places backgroundImageView first, then overlayText on top.
- StackPane.setAlignment(overlayText, Pos.BOTTOM_CENTER) aligns the overlayText to the bottom center, overriding the default center alignment for that specific node.



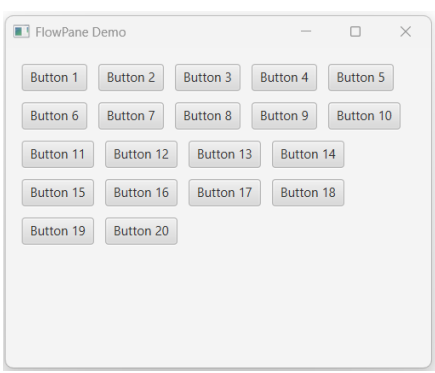
FlowPane

- **Purpose:** Arranges its children in a flow, wrapping to the next row (or column) when there isn't enough space in the current row (or column).
- **Typical Use Cases:** Image galleries, tag clouds, toolbars with many buttons that might wrap.
- **Properties:**
  - hgap, vgap: Horizontal and vertical spacing between children.
  - orientation: Orientation.HORIZONTAL (default) or Orientation.VERTICAL.
  - alignment: How the entire flow of children is aligned within the FlowPane's space.

Demo Example: Dynamic Button Flow with FlowPane

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Orientation;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.FlowPane;
import javafx.stage.Stage;

public class FlowPaneDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        FlowPane root = new FlowPane(Orientation.HORIZONTAL, 10, 10); // Horizontal, hgap=10, vgap=10
        root.setPadding(new Insets(15));
        root.setAlignment(Pos.TOP_LEFT); // Align the entire flow to the top-left

        // Add many buttons to demonstrate wrapping
        for (int i = 1; i <= 20; i++) {
            Button button = new Button("Button " + i);
            root.getChildren().add(button);
        }

        Scene scene = new Scene(root, 400, 300);
        primaryStage.setTitle("FlowPane Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

- As you resize the window, the buttons in the FlowPane will automatically wrap to the next line when space runs out, demonstrating its dynamic nature.
- root.setHgap(10) and root.setVgap(10) specify the spacing between children.
- Orientation.HORIZONTAL makes it flow horizontally (default).



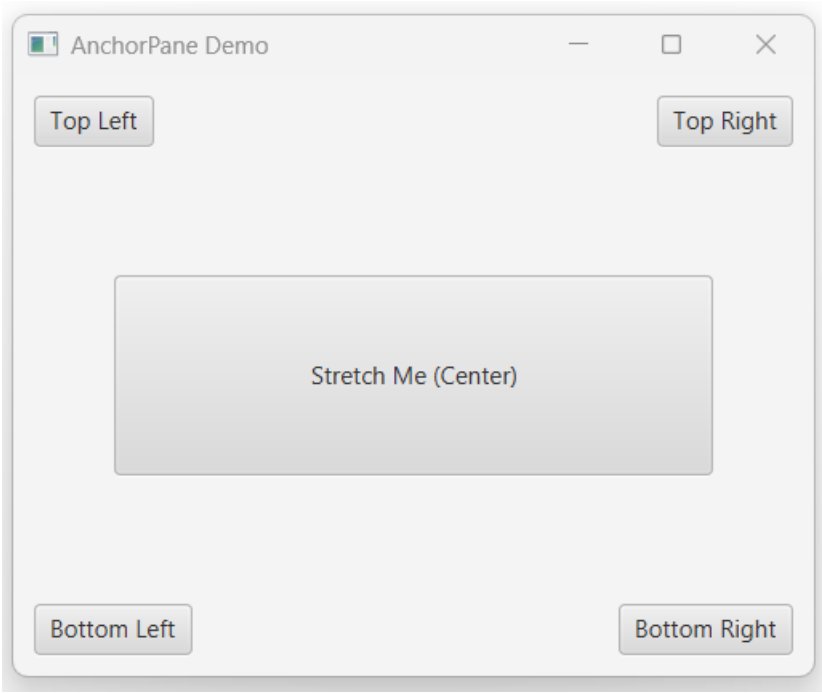
AnchorPane

- **Purpose:** Allows children to be anchored to the top, bottom, left, and/or right edges of the pane.
- **Typical Use Cases:** Placing elements at specific corners or edges, status bars, single controls that need to stay fixed relative to a window edge.
- **Properties:**
  - AnchorPane.setTopAnchor(Node child, Double value), setBottomAnchor(Node child, Double value), setLeftAnchor(Node child, Double value), setRightAnchor(Node child, Double value): Static methods to set the distance from the pane's edges.
  - If you set opposing anchors (e.g., setLeftAnchor and setRightAnchor), the child will stretch to fill the space between those anchors.

Demo Example: Anchored Buttons with AnchorPane

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.AnchorPane;
import javafx.stage.Stage;

public class AnchorPaneDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        AnchorPane root = new AnchorPane();
        root.setPrefSize(400, 300); // Set a preferred size for the pane itself

        Button topLeftBtn = new Button("Top Left");
        AnchorPane.setTopAnchor(topLeftBtn, 10.0);
        AnchorPane.setLeftAnchor(topLeftBtn, 10.0);

        Button topRightBtn = new Button("Top Right");
        AnchorPane.setTopAnchor(topRightBtn, 10.0);
        AnchorPane.setRightAnchor(topRightBtn, 10.0);

        Button bottomLeftBtn = new Button("Bottom Left");
        AnchorPane.setBottomAnchor(bottomLeftBtn, 10.0);
        AnchorPane.setLeftAnchor(bottomLeftBtn, 10.0);

        Button bottomRightBtn = new Button("Bottom Right");
        AnchorPane.setBottomAnchor(bottomRightBtn, 10.0);
        AnchorPane.setRightAnchor(bottomRightBtn, 10.0);

        Button centerStretchBtn = new Button("Stretch Me (Center)");
        AnchorPane.setTopAnchor(centerStretchBtn, 100.0);
        AnchorPane.setBottomAnchor(centerStretchBtn, 100.0);
        AnchorPane.setLeftAnchor(centerStretchBtn, 50.0);
        AnchorPane.setRightAnchor(centerStretchBtn, 50.0);

       root.getChildren().addAll(topLeftBtn, topRightBtn, bottomLeftBtn, bottomRightBtn,
           centerStretchBtn);

        Scene scene = new Scene(root);
        primaryStage.setTitle("AnchorPane Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

- The four corner buttons are anchored a fixed distance from their respective edges.
- centerStretchBtn has both left and right, and top and bottom anchors, causing it to stretch both horizontally and vertically as the window resizes, maintaining its distance from the edges.



Nesting Layouts Effectively

The true power of JavaFX layouts comes from nesting them. By combining different layout panes, you can create virtually any complex UI structure.

Example: Complex UI with Nested Layouts

Let's combine BorderPane, VBox, HBox, and GridPane to create a more sophisticated form layout.

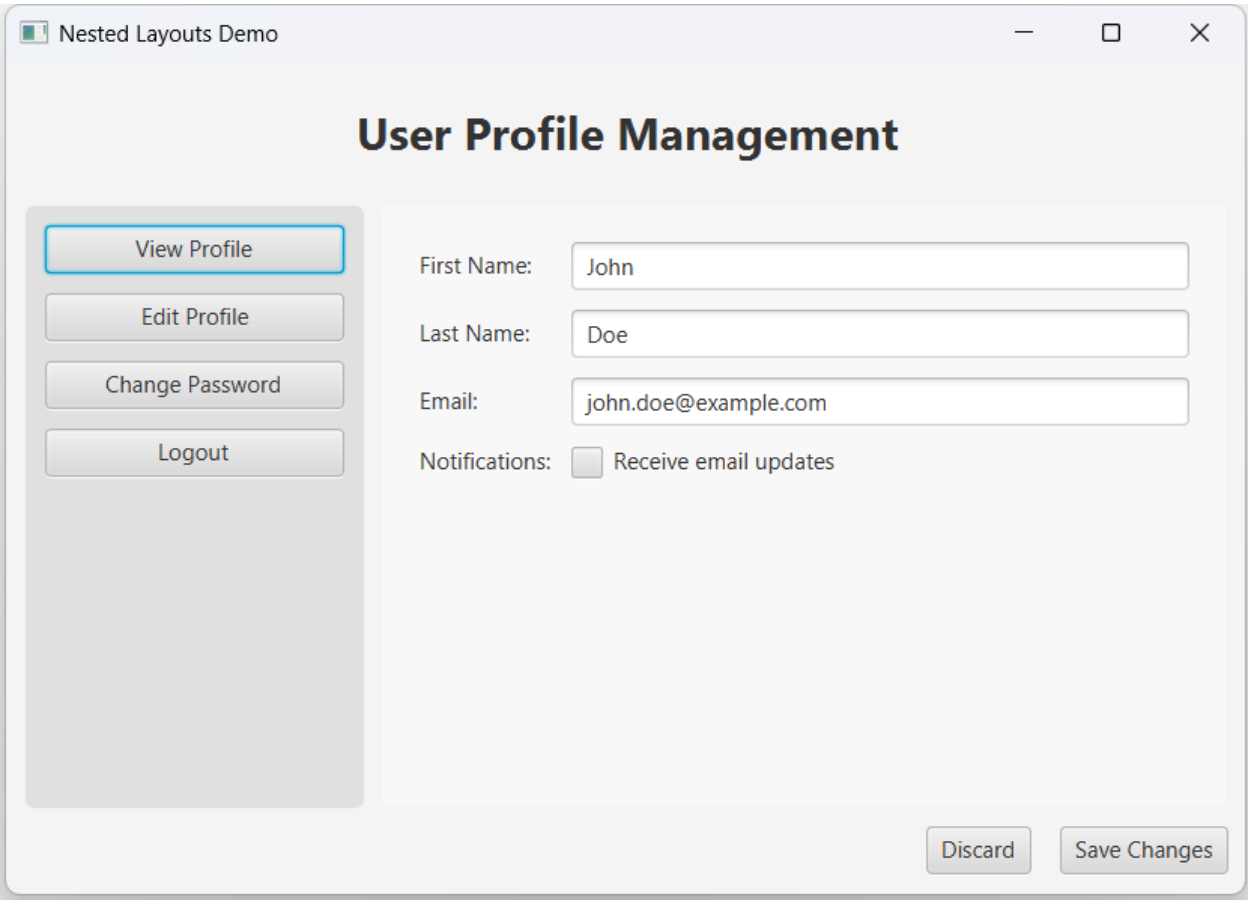
```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.CheckBox;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class NestedLayoutsDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        BorderPane mainLayout = new BorderPane();
        mainLayout.setPadding(new Insets(10));

        // Top - Title Section
        Label title = new Label("User Profile Management");
        title.setStyle("-fx-font-size: 24px; -fx-font-weight: bold; -fx-padding: 10px;");
        HBox topSection = new HBox(title);
        topSection.setAlignment(Pos.CENTER);
        mainLayout.setTop(topSection);
        BorderPane.setMargin(topSection, new Insets(0, 0, 10, 0)); // Bottom margin for title

        // Left - Navigation
        VBox navigation = new VBox(10);
        navigation.setPadding(new Insets(10));
        navigation.setStyle("-fx-background-color: #e0e0e0; -fx-background-radius: 5px;");
        navigation.setPrefWidth(180);
        navigation.getChildren().addAll(
                new Button("View Profile"),
                new Button("Edit Profile"),
                new Button("Change Password"),
                new Button("Logout")
        );
        for(javafx.scene.Node node : navigation.getChildren()){
            if(node instanceof Button) ((Button)node).setMaxWidth(Double.MAX_VALUE); // Make
buttons fill width
        }
        mainLayout.setLeft(navigation);
        BorderPane.setMargin(navigation, new Insets(0, 10, 0, 0)); // Right margin for navigation

        // Center - Profile Edit Form (using GridPane)
        GridPane profileForm = new GridPane();
        profileForm.setHgap(10);
        profileForm.setVgap(10);
        profileForm.setPadding(new Insets(20));
        profileForm.setStyle("-fx-background-color: #f8f8f8; -fx-background-radius: 5px;");

        Label firstNameLabel = new Label("First Name:");
        TextField firstNameField = new TextField("John");
        Label lastNameLabel = new Label("Last Name:");
        TextField lastNameField = new TextField("Doe");
```

```
        Label emailLabel = new Label("Email:");
        TextField emailField = new TextField("john.doe@example.com");
        Label notificationsLabel = new Label("Notifications:");
        CheckBox emailNotify = new CheckBox("Receive email updates");

        profileForm.add(firstNameLabel, 0, 0);
        profileForm.add(firstNameField, 1, 0);
        profileForm.add(lastNameLabel, 0, 1);
        profileForm.add(lastNameField, 1, 1);
        profileForm.add(emailLabel, 0, 2);
        profileForm.add(emailField, 1, 2);
        profileForm.add(notificationsLabel, 0, 3);
        profileForm.add(emailNotify, 1, 3);

        // Make input fields expand
        GridPane.setHgrow(firstNameField, javafx.scene.layout.Priority.ALWAYS);
        GridPane.setHgrow(lastNameField, javafx.scene.layout.Priority.ALWAYS);
        GridPane.setHgrow(emailField, javafx.scene.layout.Priority.ALWAYS);
        GridPane.setHgrow(emailNotify, javafx.scene.layout.Priority.ALWAYS);

        mainLayout.setCenter(profileForm);

        // Bottom - Action Buttons
        HBox actionButtons = new HBox(15);
        actionButtons.setPadding(new Insets(10, 0, 0, 0)); // Top padding
        actionButtons.setAlignment(Pos.CENTER_RIGHT);
        Button saveButton = new Button("Save Changes");
        Button discardButton = new Button("Discard");
        actionButtons.getChildren().addAll(discardButton, saveButton);
        mainLayout.setBottom(actionButtons);


        Scene scene = new Scene(mainLayout, 800, 600);
        primaryStage.setTitle("Nested Layouts Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

This example uses:

- BorderPane as the main structure.
- HBox for the TOP (title) and BOTTOM (action buttons) sections.
- VBox for the LEFT (navigation menu).
- GridPane for the CENTER (profile edit form).

By combining these, we achieve a flexible and well-organized UI.


Nested Layouts Demo
User Profile Management
View Profile
Edit Profile
Change Password
Logout
First Name:
John
Last Name:
Doe
Email:
john.doe@example.com
Notifications:
Receive email updates
Discard
Save Changes

3.3. Basic UI Controls

UI controls are the interactive elements users interact with. This section will cover the fundamental controls and, crucially, how to handle events they generate.

3.3.1. Event Handlers (MAJOR SECTION)

Event handling is the backbone of any interactive GUI application. It's how your program responds to user actions like clicks, key presses, and mouse movements. JavaFX has a powerful and consistent event handling mechanism.

Comprehensive Coverage: Event Filtering to Bubbling

In JavaFX, events propagate through the Scene Graph in two phases:

1. **Capturing/Filtering Phase (from Root to Target):**
    - When an event occurs (e.g., mouse click on a button), the event starts at the **root node** of the scene graph (e.g., the BorderPane in our previous example).
    - It then travels down the hierarchy, through parent nodes, towards the actual **event target** (the node that generated the event, e.g., the Button).
    - During this phase, you can register **event filters** (addEventFilter). Filters allow you to intercept and process an event *before* it reaches the target. A filter can consume the event, preventing it from reaching the target or further filters/handlers.
2. **Bubbling Phase (from Target to Root):**
    - After the event reaches the target node (and any filters on the target have processed it), it then travels back *up* the hierarchy, from the target node towards the root node.
    - During this phase, you can register **event handlers** (addEventHandler or convenience methods like setOnAction). Handlers process the event as it bubbles up.

Key Concepts:

- **Event:** An object that encapsulates information about an occurrence (e.g., MouseEvent, KeyEvent, ActionEvent).
- **Event Source:** The object on which the event initially occurred (e.g., the Button that was clicked).
- **Event Target:** The node in the Scene Graph where the event occurred (same as Event Source for most UI control events).
- **Event Listener/Handler:** An object that "listens" for or "handles" specific types of events. It contains the code to execute when an event occurs.
- **Event Filter:** A specific type of listener that acts during the capturing phase, allowing early interception and potential consumption of an event.
- **Event.consume():** A crucial method. When called on an Event object, it marks the event as "consumed." This prevents the event from propagating further down (in filtering) or further up (in bubbling) the Scene Graph. Use it when you want to handle an event exclusively at a particular point.

Implementation Methods:

There are several ways to implement event handling in JavaFX, each with its advantages.

1. **Anonymous Inner Classes:**
   - Traditional Java approach for implementing interfaces with a single method.
   - Can be verbose but explicit.

```
import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class EventDemoAnonymous extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button btn = new Button("Click Me (Anonymous)");

        // Anonymous inner class for event handling
        btn.setOnAction(new EventHandler<ActionEvent>() {
            @Override
            public void handle(ActionEvent event) {
                System.out.println("Button clicked using anonymous inner class!");
            }
        });

        StackPane root = new StackPane(btn);
        Scene scene = new Scene(root, 300, 200);
        primaryStage.setTitle("Anonymous Event Handler Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
}
```

2. **Separate Named Classes Implementing EventHandler<T>:**
   - Good for reusable event logic, especially if the same handling logic applies to multiple controls.
   - Encourages separation of concerns.

```java
import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

// Separate named class for handling button clicks
class MyButtonHandler implements EventHandler<ActionEvent> {
    private String message;

    public MyButtonHandler(String message) {
        this.message = message;
    }

    @Override
    public void handle(ActionEvent event) {
      System.out.println(message + " - Event source: " +
      event.getSource().getClass().getSimpleName());
    }
}

public class EventDemoNamedClass extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button btn1 = new Button("Click Me 1 (Named Class)");
        btn1.setOnAction(new MyButtonHandler("Button 1 clicked"));

        Button btn2 = new Button("Click Me 2 (Named Class)");
        btn2.setOnAction(new MyButtonHandler("Button 2 clicked"));

        VBox root = new VBox(10, btn1, btn2);
        root.setAlignment(Pos.CENTER);
        Scene scene = new Scene(root, 300, 200);
        primaryStage.setTitle("Named Class Event Handler Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
}
```

3. **Using Lambda Expressions:**
   - **Highly Recommended & Preferred:** The most concise and modern way to handle events in JavaFX since Java 8.
   - Works because EventHandler<T> is a **functional interface** (an interface with a single abstract method).

```java
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class EventDemoLambda extends Application {
    @Override
    public void start(Stage primaryStage) {
        Button btnHello = new Button("Say Hello");
        Button btnGoodbye = new Button("Say Goodbye");

        // Lambda expression for event handling (most common and concise)
        btnHello.setOnAction(event -> {
            System.out.println("Hello there!");
            // You can access other UI elements here if needed
        });

        btnGoodbye.setOnAction(e -> System.out.println("Goodbye for now!")); // Even shorter lambda

        VBox root = new VBox(10, btnHello, btnGoodbye);
        root.setAlignment(Pos.CENTER);
        Scene scene = new Scene(root, 300, 200);
        primaryStage.setTitle("Lambda Event Handler Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
}
```

- **Creativity in Lambda Examples:**
  - **Dynamic UI Update:**

```java
Label statusLabel = new Label("No action yet.");
Button updateButton = new Button("Update Status");
updateButton.setOnAction(event -> {
    statusLabel.setText("Status updated at " + LocalTime.now());
    // Accessing and modifying another UI element within the lambda
});
// ... add to scene
```

  - **Input Validation on the Fly:** (See TextField example later)
  - **Simple Calculator Logic:**

```java
TextField num1 = new TextField();
TextField num2 = new TextField();
Button addButton = new Button("Add");
Label resultLabel = new Label("Result:");

addButton.setOnAction(event -> {
    try {
        double n1 = Double.parseDouble(num1.getText());
        double n2 = Double.parseDouble(num2.getText());
        resultLabel.setText("Result: " + (n1 + n2));
    } catch (NumberFormatException ex) {
        resultLabel.setText("Invalid input!");
    }
});
// ... add to scene
```

addEventHandler() vs. setOnAction()/setOnMouseClicked()/etc.

This is a critical distinction!

- **Convenience Methods (setOnAction, setOnMouseClicked, setOnKeyPressed, etc.):**
  - These are specific methods provided by nodes (especially controls) for their *primary* or most common event type.
  - They register a handler for the **bubbling phase** only.
  - They effectively set the *sole* handler for that event type. If you call setOnAction twice, the second call *replaces* the first.
  - Often the simplest choice for direct control responses.
- **addEventHandler(EventType<T> eventType, EventHandler<? super T> handler):**
  - Registers a handler for the **bubbling phase**.
  - You can register *multiple* handlers for the same event type on a single node. All registered handlers will be invoked in the order they were added.
  - Requires you to specify the EventType (e.g., MouseEvent.MOUSE_CLICKED, KeyEvent.KEY_PRESSED).
  - More flexible when you need multiple independent responses to the same event.
- **addEventFilter(EventType<T> eventType, EventHandler<? super T> filter):**
  - Registers a handler for the **capturing/filtering phase**.
  - Allows you to intercept events *before* they reach the target's handlers.
  - Crucial for situations where you want to validate input, prevent default behavior, or implement global event listeners.
  - Can also register multiple filters.

When to use which:

- Use **convenience methods** (setOnAction, etc.) for simple, single-response actions directly tied to a control's primary event.
- Use **addEventHandler** when you need multiple, independent pieces of code to react to the same event on a node, all during the bubbling phase.
- Use **addEventFilter** when you need to intercept an event early, potentially consume it, or modify it before it reaches its target or normal handlers. This is essential for features like input validation, global shortcuts, or drag-and-drop scenarios.

Demo: addEventHandler vs setOnAction & Event Filtering

```
import javafx.application.Application;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;
import javafx.scene.input.MouseEvent;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
```

```java
public class EventPropagationDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(20);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.CENTER);
        root.setStyle("-fx-border-color: blue; -fx-border-width: 2px;");

        Label clickLabel = new Label("Click within the blue border!");
        clickLabel.setStyle("-fx-font-size: 16px;");

        Button button = new Button("Click Me!");
        button.setPrefSize(120, 40);

        // 1. Using setOnAction (convenience method)
        button.setOnAction(e -> {
            System.out.println("  Button: setOnAction fired! (Bubbling)");
        });

        // 2. Using addEventHandler (for ActionEvent) - will also fire in bubbling phase
        button.addEventHandler(ActionEvent.ACTION, e -> {
            System.out.println("  Button: addEventHandler(ActionEvent) fired! (Bubbling)");
        });

        // 3. Using addEventHandler for MouseEvent on the button (bubbling phase)
        button.addEventHandler(MouseEvent.MOUSE_CLICKED, e -> {
            System.out.println("  Button: addEventHandler(MOUSE_CLICKED) fired! (Bubbling)");
            // Example: Consume mouse click on button if Ctrl is held down
            if (e.isControlDown()) {
                System.out.println("  Button: Ctrl + Click detected. Consuming mouse event!");
                e.consume(); // Prevents this mouse click from bubbling further up
            }
        });

        // 4. Registering a filter on the root VBox (capturing phase)
        root.addEventFilter(MouseEvent.MOUSE_CLICKED, e -> {
            System.out.println("Root VBox: Event Filter - MOUSE_CLICKED (Capturing)");
            if (e.getClickCount() == 2) { // If it's a double click
                System.out.println("Root VBox: Double click detected! Consuming event.");
                e.consume(); // Consume the event, preventing it from reaching children or bubbling
            }
        });

        // 5. Registering a handler on the root VBox (bubbling phase)
        root.addEventHandler(MouseEvent.MOUSE_CLICKED, e -> {
            System.out.println("Root VBox: Event Handler - MOUSE_CLICKED (Bubbling)");
            if (!e.isConsumed()) { // Only react if not consumed by a filter or child handler
                System.out.println("Root VBox: Click processed by root handler.");
            }
        });

        // 6. Keyboard event handler on the scene
        Scene scene = new Scene(root, 400, 300);
        scene.addEventHandler(KeyEvent.KEY_PRESSED, e -> {
            if (e.getCode() == KeyCode.SPACE) {
                System.out.println("Scene: Space bar pressed!");
                e.consume(); // Consume to prevent default behaviors (e.g., button activation)
            } else if (e.isShiftDown() && e.getCode() == KeyCode.C) {
                System.out.println("Scene: Shift + C pressed!");
            }
        });

        root.getChildren().addAll(clickLabel, button);

        primaryStage.setTitle("Event Propagation & Filtering Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
```

```
    public static void main(String[] args) {
        launch(args);
    }
}
```

Experiment with the above code:

- **Single click the button:** You'll see setOnAction, addEventHandler(ActionEvent), and addEventHandler(MOUSE_CLICKED) on the button fire, then addEventHandler(MOUSE_CLICKED) on the root VBox fire during bubbling.
- **Ctrl + Click the button:** The addEventHandler(MOUSE_CLICKED) on the button will consume the event. You'll see its output, but the root VBox's handler for MOUSE_CLICKED will *not* fire.
- **Double click anywhere in the blue area (including button):** The root VBox's addEventFilter(MOUSE_CLICKED) will detect the double click and consume it. *No other handlers or filters further down or up the hierarchy for that specific mouse event will fire.*
- **Press Space or Shift+C when the window is active:** The scene's KeyEvent handler will catch it. Space is consumed.

Event Types

JavaFX provides a comprehensive set of event classes, typically found in javafx.event and javafx.scene.input packages.

- **ActionEvent:**
  - Raised by controls that perform an "action" (e.g., Button click, TextField enter press, MenuItem selection).
- **MouseEvent:**
  - Generated by mouse actions (click, press, release, move, drag, enter, exit).
  - Subtypes: MOUSE_CLICKED, MOUSE_PRESSED, MOUSE_RELEASED, MOUSE_MOVED, MOUSE_DRAGGED, MOUSE_ENTERED, MOUSE_EXITED.
  - Methods: getX(), getY() (coordinates), getButton() (which mouse button), getClickCount(), isShiftDown(), isControlDown(), etc.
- **KeyEvent:**
  - Generated by keyboard actions (key press, key release, key typed).
  - Subtypes: KEY_PRESSED, KEY_RELEASED, KEY_TYPED.
  - Methods: getCode() (the specific key, e.g., KeyCode.ENTER), getText() (the character generated), isShiftDown(), isControlDown(), etc.
- **InputMethodEvent:**
  - Generated by input method changes (e.g., for Asian languages).
- **DragEvent:** For drag-and-drop operations.
- **ScrollEvent:** For mouse wheel scrolling.
- **TouchEvent:** For touch screen interactions.
- **WindowEvent:** For events related to a Stage (e.g., close request, showing, hiding).

Which Controls Raise What (Primary Events)?? Assignment!!

Common Mistakes in Event Handling:

- **Forgetting to Register Handlers:** Creating the EventHandler code but not actually setting it on the control (e.g., creating a lambda but not calling btn.setOnAction(...)).
- **Incorrect Event Types:** Registering for MouseEvent.MOUSE_CLICKED when you meant ActionEvent for a button. Buttons primarily fire ActionEvent.
- **Blocking the JavaFX Application Thread:** Performing long-running operations directly inside an event handler. This will freeze your UI. For heavy tasks, use Platform.runLater() to update the UI from a background thread.
- **Not Consuming Events:** Allowing events to propagate when you intended to handle them exclusively, leading to unexpected behavior in parent nodes.
- **Replacing Handlers:** Accidentally overwriting a handler set by setOnAction() by calling it again. Use addEventHandler() if you need multiple handlers.

Recommended Approaches & Tricks:

- **Lambda Expressions:** Embrace them for conciseness and readability for most common scenarios.
- **Separate Methods:** For complex event logic, define a private method in your controller class and reference it in your lambda: button.setOnAction(this::handleButtonClick); (where handleButtonClick is a private void handleButtonClick(ActionEvent event) method).
- **Centralized Event Handling:** For applications with many similar controls, consider a single EventHandler instance that can be applied to multiple controls.

- **Debug with System.out.println:** When tracing event propagation, print messages in your filters and handlers to understand the order of execution.
- **Keyboard Shortcuts:** Use Scene.addEventHandler(KeyEvent.KEY_PRESSED, ...) for global keyboard shortcuts (e.g., Ctrl+S for save). Remember to consume() the event if it's meant to be exclusive.
- **Focus Management:** Be aware of which control has focus, as KeyEvents often go to the focused node first.

Demo Examples for Event Handling

Let's put these concepts into practice with more elaborate examples.

Demo 1: Button with Multi-faceted Interaction (Mouse, Keyboard, Context)

```
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.input.KeyCode;
import javafx.scene.input.MouseButton;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class AdvancedButtonEvents extends Application {

    private Label statusLabel = new Label("Interact with the button!");

    @Override
    public void start(Stage primaryStage) {
        Button myButton = new Button("Smart Button");
        myButton.setPrefSize(150, 50);

        // 1. Basic ActionEvent on click (Left Mouse Button)
        myButton.setOnAction(e -> {
            statusLabel.setText("Action: Button was activated!");
            System.out.println("--- ActionEvent triggered ---");
        });

        // 2. MouseEvent for Right-Click (Context Menu Simulation)
        myButton.addEventHandler(MouseEvent.MOUSE_CLICKED, e -> {
            if (e.getButton() == MouseButton.SECONDARY) { // Right click
                statusLabel.setText("Context: Right-clicked!");
                System.out.println("--- Right-Click MouseEvent triggered ---");
                // In a real app, you'd show a ContextMenu here
            } else if (e.getButton() == MouseButton.PRIMARY && e.isShiftDown()) { // Shift + Left Click
                statusLabel.setText("Special: Shift + Left-clicked!");
                System.out.println("--- Shift + Left-Click MouseEvent triggered ---");
                e.consume(); // Consume to prevent the setOnAction from firing
            }
        });

        // 3. KeyEvent for Enter (same as ActionEvent), but also Tab/Space
        myButton.setOnKeyPressed(e -> {
            if (e.getCode() == KeyCode.ENTER) {
                statusLabel.setText("Keyboard: Enter pressed!");
                System.out.println("--- KeyEvent: Enter pressed ---");
            } else if (e.getCode() == KeyCode.SPACE) {
                statusLabel.setText("Keyboard: Space pressed!");
                System.out.println("--- KeyEvent: Space pressed ---");
            }
            // You can consume if you want to override default behavior
            // e.consume();
        });

        VBox root = new VBox(20, myButton, statusLabel);
        root.setAlignment(Pos.CENTER);
        Scene scene = new Scene(root, 400, 250);
        primaryStage.setTitle("Advanced Button Event Handling");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Demo 2: Text Field Input Validation on Focus Loss and Real-time Character Filtering

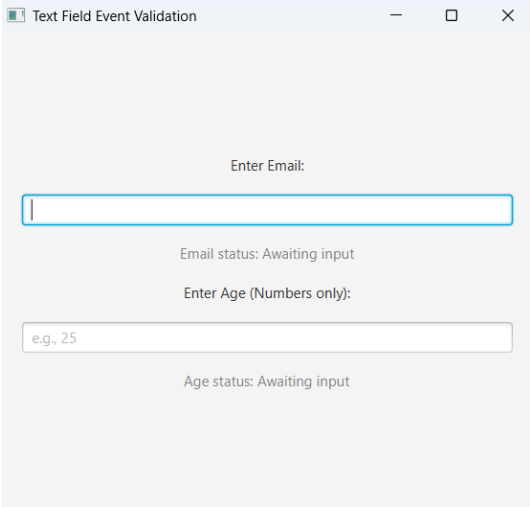
This example demonstrates two different event handling strategies for a TextField:

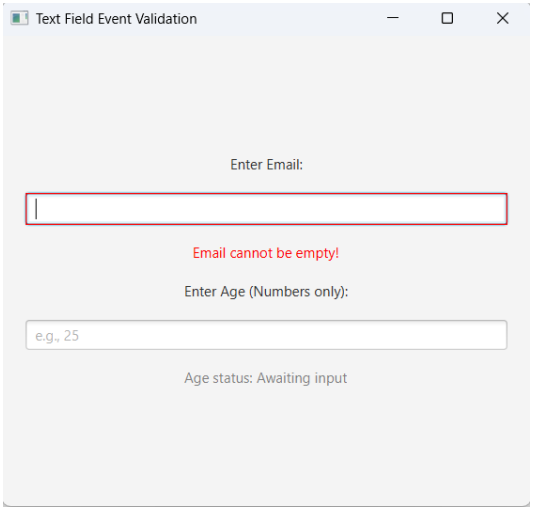
- **Focus Listener:** Validates the input when the TextField loses focus (focusProperty().addListener(...)). This is a common pattern for "on blur" validation.
- **Event Filter (Key Typed):** Prevents invalid characters from even being entered (addEventFilter(KeyEvent.KEY_TYPED, ...)) for a numeric-only field.

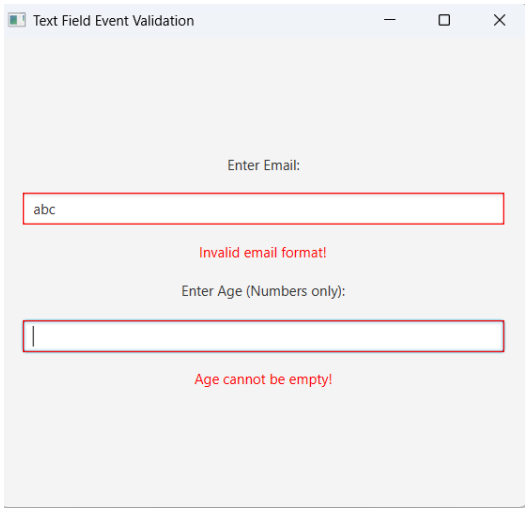
```
import javafx.application.Application;
import javafx.beans.value.ChangeListener;
import javafx.beans.value.ObservableValue;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.input.KeyEvent;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.stage.Stage;

public class TextFieldValidationDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(15);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.CENTER);

        Label emailPrompt = new Label("Enter Email:");
        TextField emailField = new TextField();
        emailField.setPromptText("example@domain.com");
        Label emailStatus = new Label("Email status: Awaiting input");
        emailStatus.setTextFill(Color.GRAY);

        // --- Email Validation on Focus Loss ---
        // ChangeListener for focusProperty()
        emailField.focusedProperty().addListener(new ChangeListener<Boolean>() {
            @Override
            public void changed(ObservableValue<? extends Boolean> observable, Boolean oldValue,
Boolean newValue) {
                if (!newValue) { // oldValue was true, newValue is false => focus lost
                    String email = emailField.getText();
                    if (email.isEmpty()) {
                        emailStatus.setText("Email cannot be empty!");
                        emailStatus.setTextFill(Color.RED);
                        emailField.setStyle("-fx-border-color: red;");
                    } else if (!email.matches("^[\\w!#$%&'*+/=?`{|}~^-
]+(?:\\.[\\w!#$%&'*+/=?`{|}~^-]+)*@(?:[a-zA-Z0-9-]+\\.)+[a-zA-Z]{2,6}$")) {
                        emailStatus.setText("Invalid email format!");
                        emailStatus.setTextFill(Color.RED);
                        emailField.setStyle("-fx-border-color: red;");
                    } else {
                        emailStatus.setText("Email looks valid.");
                        emailStatus.setTextFill(Color.GREEN);
                        emailField.setStyle(null); // Clear custom style
                    }
                }
            }
        });


        Label agePrompt = new Label("Enter Age (Numbers only):");
        TextField ageField = new TextField();
```

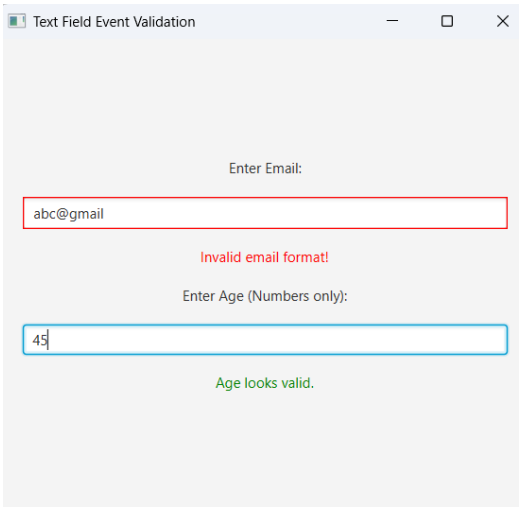
```
        ageField.setPromptText("e.g., 25");
        Label ageStatus = new Label("Age status: Awaiting input");
        ageStatus.setTextFill(Color.GRAY);

        // --- Numeric Input Filtering using addEventFilter (KEY_TYPED) ---
        ageField.addEventFilter(KeyEvent.KEY_TYPED, event -> {
            String character = event.getCharacter();
            if (!character.matches("[0-9]") && !character.isEmpty()) { // Only allow digits
                ageStatus.setText("Only numbers are allowed for age!");
                ageStatus.setTextFill(Color.ORANGE);
                event.consume(); // Crucially, consume the event to prevent the character from being typed
            } else {
                ageStatus.setText("Age status: Typing...");
                ageStatus.setTextFill(Color.GRAY);
            }
        });

        // Additional validation for age (e.g., range) on focus loss
        ageField.focusedProperty().addListener((obs, oldVal, newVal) -> {
            if (!newVal) { // Focus lost
                String ageText = ageField.getText();
                if (ageText.isEmpty()) {
                    ageStatus.setText("Age cannot be empty!");
                    ageStatus.setTextFill(Color.RED);
                    ageField.setStyle("-fx-border-color: red;");
                } else {
                    try {
                        int age = Integer.parseInt(ageText);
                        if (age < 0 || age > 120) { // Example age range
                            ageStatus.setText("Age must be between 0 and 120.");
                            ageStatus.setTextFill(Color.RED);
                            ageField.setStyle("-fx-border-color: red;");
                        } else {
                            ageStatus.setText("Age looks valid.");
                            ageStatus.setTextFill(Color.GREEN);
                            ageField.setStyle(null);
                        }
                    } catch (NumberFormatException e) {
                        ageStatus.setText("Invalid number format for age!");
                        ageStatus.setTextFill(Color.RED);
                        ageField.setStyle("-fx-border-color: red;");
                    }
                }
            }
        });


        root.getChildren().addAll(
                emailPrompt, emailField, emailStatus,
                agePrompt, ageField, ageStatus
        );

        Scene scene = new Scene(root, 450, 400);
        primaryStage.setTitle("Text Field Event Validation");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

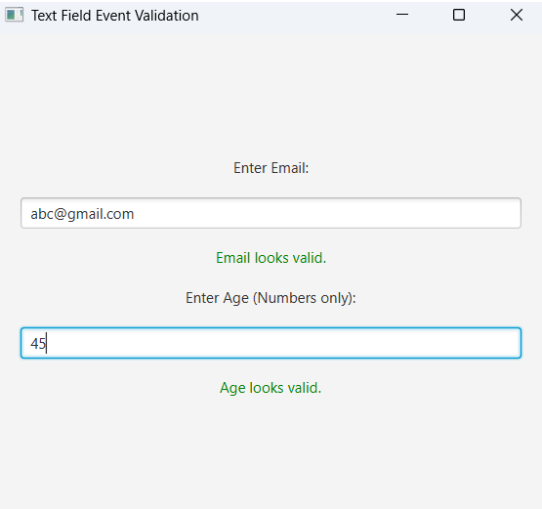
- **focusedProperty().addListener(...):** This is a ChangeListener for an ObservableValue<Boolean>. It's perfect for reacting when a control gains or loses keyboard focus.
- **addEventFilter(KeyEvent.KEY_TYPED, ...) with event.consume():** This is a powerful technique to prevent unwanted input *before* it even appears in the text field, providing immediate feedback and a better user experience for constrained input.


Text Field Event Validation
Enter Email:
Email status: Awaiting input
Enter Age (Numbers only):
e.g., 25
Age status: Awaiting input


Text Field Event Validation
Enter Email:
Email cannot be empty!
Enter Age (Numbers only):
e.g., 25
Age status: Awaiting input


Text Field Event Validation
Enter Email:
abc
Invalid email format!
Enter Age (Numbers only):
Age cannot be empty!


Text Field Event Validation
Enter Email:
abc@gmail
Invalid email format!
Enter Age (Numbers only):
45
Age looks valid.


Text Field Event Validation
Enter Email:
abc@gmail.com
Email looks valid.
Enter Age (Numbers only):
45
Age looks valid.

3.3.2. UI Controls (Basic)

Now that we understand event handling, let's explore common basic UI controls in detail, integrating event responses.

Button

- **Purpose:** Triggers an action when clicked.
- **Common Properties:** text, graphic (for an icon), disable, defaultButton (makes it react to Enter key), cancelButton (makes it react to Esc key).
- **Events:** Primarily ActionEvent (handled by setOnAction).

Demo: Button with Icon and State

```
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class ButtonDemo extends Application {

    private Label statusLabel = new Label("Button is ready.");
    private boolean buttonDisabled = false;

    @Override
    public void start(Stage primaryStage) {
        // Create an ImageView for an icon
        // You'll need a "play.png" file in your resources or project root
        // For demo, let's use a common system icon URL
        ImageView playIcon = new ImageView(new Image("https://cdn-icons-
              png.flaticon.com/512/72/72573.png"));
        playIcon.setFitWidth(16);
        playIcon.setFitHeight(16);

        Button actionButton = new Button("Start Process", playIcon); // Button with text and graphic
        actionButton.setStyle("-fx-font-size: 14px; -fx-padding: 10px 20px;");

        // Event handler for the button
        actionButton.setOnAction(e -> {
            if (!buttonDisabled) {
                statusLabel.setText("Process started! (This message will change every 2 seconds)");
                System.out.println("Process started!");
                actionButton.setText("Running...");
                actionButton.setDisable(true); // Disable button once clicked
                buttonDisabled = true;

              // Simulate a long-running process with a delay (will be covered more in concurrency)
                new Thread(() -> {
                    try {
                        Thread.sleep(2000); // Wait for 2 seconds
                    } catch (InterruptedException ex) {
                        Thread.currentThread().interrupt();
                    }
                    // Update UI on JavaFX Application Thread
                    javafx.application.Platform.runLater(() -> {
                        statusLabel.setText("Process finished.");
                        actionButton.setText("Restart Process");
                        actionButton.setDisable(false);
                        buttonDisabled = false;
                    });
                }).start();

            } else {
```

```
                statusLabel.setText("Button is currently disabled.");
            }
        });


        Button toggleDisableButton = new Button("Toggle Action Button State");
        toggleDisableButton.setOnAction(e -> {
            actionButton.setDisable(!actionButton.isDisable());
            buttonDisabled = actionButton.isDisable();
            statusLabel.setText("Action button is now " + (buttonDisabled ? "DISABLED" :
"ENABLED"));
        });


        VBox root = new VBox(20, actionButton, toggleDisableButton, statusLabel);
        root.setAlignment(Pos.CENTER);
        Scene scene = new Scene(root, 400, 300);
        primaryStage.setTitle("Button Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Label

- **Purpose:** Displays non-editable text or graphics.
- **Common Properties:** text, graphic, contentDisplay (how text and graphic are arranged), wrapText.
- **Events:** Typically doesn't generate ActionEvent, but MouseEvents can be attached for interactive labels.

Demo: Label with Wrapped Text and Graphic

```
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.ContentDisplay;
import javafx.scene.control.Label;
import javafx.scene.image.Image;
import javafx.scene.image.ImageView;
import javafx.scene.layout.VBox;
import javafx.scene.text.TextAlignment;
import javafx.stage.Stage;

public class LabelDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        // Simple text label
        Label simpleLabel = new Label("This is a simple label.");
        simpleLabel.setStyle("-fx-font-size: 16px; -fx-text-fill: #333;");

        // Label with long text that wraps
        Label wrappedLabel = new Label(
            "This is a much longer label text that needs to wrap " +
            "around to multiple lines because the available space is limited. " +
            "Labels are great for displaying descriptive information to the user."
        );
        wrappedLabel.setWrapText(true); // Enable text wrapping
        wrappedLabel.setMaxWidth(200); // Limit width to demonstrate wrapping
        wrappedLabel.setTextAlignment(TextAlignment.CENTER);
        wrappedLabel.setStyle("-fx-font-size: 14px; -fx-text-fill: #555; -fx-background-color:
#f0f0f0; -fx-padding: 10;");


        // Label with an image and text
```

```
        ImageView infoIcon = new ImageView(new Image("https://cdn-icons-
png.flaticon.com/512/3074/3074043.png"));
        infoIcon.setFitWidth(24);
        infoIcon.setFitHeight(24);

        Label iconLabel = new Label("Important Information", infoIcon);
        iconLabel.setContentDisplay(ContentDisplay.LEFT); // Icon on the left of text
        iconLabel.setStyle("-fx-font-size: 18px; -fx-text-fill: #007bff; -fx-font-weight: bold;");

        // Label with graphic above text
        ImageView cloudIcon = new ImageView(new Image("https://cdn-icons-
png.flaticon.com/512/1164/1164958.png"));
        cloudIcon.setFitWidth(32);
        cloudIcon.setFitHeight(32);
        Label cloudLabel = new Label("Cloud Status: OK", cloudIcon);
        cloudLabel.setContentDisplay(ContentDisplay.TOP); // Icon above text
        cloudLabel.setStyle("-fx-font-size: 14px; -fx-text-fill: green;");


        VBox root = new VBox(20, simpleLabel, wrappedLabel, iconLabel, cloudLabel);
        root.setAlignment(Pos.CENTER);
        Scene scene = new Scene(root, 300, 450);
        primaryStage.setTitle("Label Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

TextField

- **Purpose:** Allows single-line text input from the user.
- **Common Properties:** text, promptText (placeholder), editable, length (max length, not directly a property but can be managed).
- **Events:** ActionEvent (when Enter is pressed), KeyEvent (for individual key presses), textProperty().addListener() (for real-time changes).

Demo: TextField with promptText and textProperty Listener

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.stage.Stage;

public class TextFieldPropertiesDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(15);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.CENTER);

        Label inputLabel = new Label("Your Name:");
        TextField nameField = new TextField();
        nameField.setPromptText("Enter your full name here"); // Placeholder text
        nameField.setMaxWidth(250); // Limit preferred width

        Label displayLabel = new Label("Hello, [Name]!");
```

```
        displayLabel.setStyle("-fx-font-size: 18px; -fx-font-weight: bold; -fx-text-fill:
#4CAF50;");

        // --- Real-time update using textProperty listener ---
        // This listener reacts every time the text in the TextField changes
        nameField.textProperty().addListener((observable, oldValue, newValue) -> {
            if (newValue == null || newValue.trim().isEmpty()) {
                displayLabel.setText("Hello, [Name]!");
                displayLabel.setTextFill(Color.GRAY);
            } else {
                displayLabel.setText("Hello, " + newValue + "!");
                displayLabel.setTextFill(Color.BLUE);
            }
        });

        // --- ActionEvent on Enter press ---
        nameField.setOnAction(e -> {
            System.out.println("Enter pressed in nameField. Current text: " + nameField.getText());
            // You could trigger a form submission here, for instance
        });

        root.getChildren().addAll(inputLabel, nameField, displayLabel);

        Scene scene = new Scene(root, 350, 250);
        primaryStage.setTitle("TextField Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

PasswordField

- **Purpose:** Similar to TextField, but obscures input characters (shows asterisks or dots).
- **Common Properties:** Same as TextField (text, promptText, editable).
- **Events:** Same as TextField.
- **Note:** The actual password can still be retrieved using getText(). For security, always handle passwords carefully (e.g., hashing before storage/transmission).

Demo: Simple Login with PasswordField

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.PasswordField;
import javafx.scene.control.TextField;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.HBox;
import javafx.scene.paint.Color;
import javafx.stage.Stage;

public class PasswordFieldDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        GridPane grid = new GridPane();
        grid.setAlignment(Pos.CENTER);
        grid.setHgap(10);
```

```
        grid.setVgap(10);
        grid.setPadding(new Insets(25, 25, 25, 25));

        Label userName = new Label("User Name:");
        grid.add(userName, 0, 1);

        TextField userTextField = new TextField();
        grid.add(userTextField, 1, 1);

        Label pw = new Label("Password:");
        grid.add(pw, 0, 2);

        PasswordField pwBox = new PasswordField(); // This is the key difference
        grid.add(pwBox, 1, 2);

        Button signInBtn = new Button("Sign In");
        HBox hbBtn = new HBox(10);
        hbBtn.setAlignment(Pos.BOTTOM_RIGHT);
        hbBtn.getChildren().add(signInBtn);
        grid.add(hbBtn, 1, 4);

        final Label actiontarget = new Label();
        grid.add(actiontarget, 1, 6);

        signInBtn.setOnAction(e -> {
            String username = userTextField.getText();
            String password = pwBox.getText(); // Get the entered password
            System.out.println("Attempting login with: " + username + " / " + password);

            // Simple validation (DO NOT use for real systems!)
            if ("admin".equals(username) && "secret".equals(password)) {
                actiontarget.setText("Login Successful!");
                actiontarget.setTextFill(Color.GREEN);
            } else {
                actiontarget.setText("Invalid username or password.");
                actiontarget.setTextFill(Color.RED);
            }
            pwBox.clear(); // Clear password field after attempt
        });

        Scene scene = new Scene(grid, 350, 275);
        primaryStage.setTitle("PasswordField Demo (Login)");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

TextArea

- **Purpose:** Allows multi-line text input and display.
- **Common Properties:** text, promptText, editable, wrapText (automatically wraps long lines).
- **Events:** Similar to TextField (textProperty() for changes, KeyEvent for input). No ActionEvent on Enter, as Enter creates a new line.

Demo: TextArea with wrapText and Character Count

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.TextArea;
import javafx.scene.layout.VBox;
```

```
import javafx.stage.Stage;

public class TextAreaDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(15);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.CENTER);

        Label headerLabel = new Label("Enter Your Feedback:");
        headerLabel.setStyle("-fx-font-size: 16px; -fx-font-weight: bold;");

        TextArea feedbackArea = new TextArea();
        feedbackArea.setPromptText("Type your feedback here...");
        feedbackArea.setWrapText(true); // Crucial for multi-line text areas
        feedbackArea.setPrefRowCount(8); // Suggest initial number of rows
        feedbackArea.setPrefColumnCount(30); // Suggest initial number of columns (width)
        feedbackArea.setMaxWidth(400); // Max width to constrain it

        Label charCountLabel = new Label("Characters: 0");

        // --- Real-time character count using textProperty listener ---
        feedbackArea.textProperty().addListener((observable, oldValue, newValue) -> {
            int count = (newValue == null) ? 0 : newValue.length();
            charCountLabel.setText("Characters: " + count);
            if (count > 200) { // Example: Max characters allowed
                charCountLabel.setStyle("-fx-text-fill: red;");
            } else {
                charCountLabel.setStyle("-fx-text-fill: black;");
            }
        });

        root.getChildren().addAll(headerLabel, feedbackArea, charCountLabel);

        Scene scene = new Scene(root, 500, 400);
        primaryStage.setTitle("TextArea Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

CheckBox

- **Purpose:** Allows users to select or deselect a single option. Can be independent of other checkboxes.
- **Common Properties:** text, selected (boolean state), allowIndeterminate (for three-state checkboxes).
- **Events:** ActionEvent (when state changes), selectedProperty().addListener() (for reacting to state changes).

Demo: CheckBox for Preferences

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.CheckBox;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class CheckBoxDemo extends Application {
```

```
    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(15);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.TOP_LEFT);

        Label header = new Label("User Preferences:");
        header.setStyle("-fx-font-size: 18px; -fx-font-weight: bold;");

        CheckBox emailNotification = new CheckBox("Receive email notifications");
        CheckBox smsNotification = new CheckBox("Receive SMS alerts");
        CheckBoxdarkMode = new CheckBox("Enable Dark Mode");
        darkMode.setAllowIndeterminate(true); // Example of a three-state checkbox

        Label statusLabel = new Label("Notifications: Email OFF, SMS OFF. Dark Mode: UNKNOWN");

        // --- Event Handling for CheckBoxes ---
        // Using setOnAction (ActionEvent)
        emailNotification.setOnAction(e -> {
            updateStatus(emailNotification, smsNotification, darkMode, statusLabel);
            System.out.println("Email Notifications: " + emailNotification.isSelected());
        });

        // Using selectedProperty listener
        smsNotification.selectedProperty().addListener((observable, oldValue, newValue) -> {
            updateStatus(emailNotification, smsNotification, darkMode, statusLabel);
            System.out.println("SMS Alerts: " + newValue);
        });

        // For indeterminate state, check isIndeterminate()
        darkMode.setOnAction(e -> {
            updateStatus(emailNotification, smsNotification, darkMode, statusLabel);
            if (darkMode.isIndeterminate()) {
                System.out.println("Dark Mode: Indeterminate");
            } else {
                System.out.println("Dark Mode: " + (darkMode.isSelected() ? "Enabled" :
"Disabled"));
            }
        });


        root.getChildren().addAll(header, emailNotification, smsNotification, darkMode, new
Label(), statusLabel); // Empty label for spacing

        Scene scene = new Scene(root, 400, 300);
        primaryStage.setTitle("CheckBox Demo");
        primaryStage.setScene(scene);
        primaryStage.show();

        // Initial status update
        updateStatus(emailNotification, smsNotification, darkMode, statusLabel);
    }

    private void updateStatus(CheckBox email, CheckBox sms, CheckBox dark, Label status) {
        String emailState = email.isSelected() ? "ON" : "OFF";
        String smsState = sms.isSelected() ? "ON" : "OFF";
        String darkState;
        if (dark.isIndeterminate()) {
            darkState = "UNKNOWN";
        } else {
            darkState = dark.isSelected() ? "ENABLED" : "DISABLED";
        }
        status.setText(String.format("Notifications: Email %s, SMS %s. Dark Mode: %s", emailState,
smsState, darkState));
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

RadioButton (with ToggleGroup)

- **Purpose:** Allows users to select *only one* option from a mutually exclusive group of choices.
- **Key Concept:** ToggleGroup. All RadioButtons that belong to the same ToggleGroup will behave as a single choice set. Selecting one automatically deselects the others in the group.
- **Common Properties:** text, selected (only one in a group can be true), toggleGroup.
- **Events:** ActionEvent (when selection changes), toggleGroup.selectedToggleProperty().addListener() (recommended for group changes).

Demo: RadioButton for Payment Method Selection

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.RadioButton;
import javafx.scene.control.Toggle;
import javafx.scene.control.ToggleGroup;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class RadioButtonDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(15);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.TOP_LEFT);

        Label header = new Label("Select Payment Method:");
        header.setStyle("-fx-font-size: 18px; -fx-font-weight: bold;");

        ToggleGroup paymentGroup = new ToggleGroup(); // Crucial: Create a ToggleGroup

        RadioButton creditCard = new RadioButton("Credit Card");
        creditCard.setToggleGroup(paymentGroup); // Assign to the group
        creditCard.setSelected(true); // Default selection

        RadioButton paypal = new RadioButton("PayPal");
        paypal.setToggleGroup(paymentGroup);

        RadioButton bankTransfer = new RadioButton("Bank Transfer");
        bankTransfer.setToggleGroup(paymentGroup);

        Label selectionStatus = new Label("Selected: Credit Card");

        // --- Event Handling for ToggleGroup Selection Change ---
        // Listen to the selectedToggleProperty of the ToggleGroup
        paymentGroup.selectedToggleProperty().addListener((observable, oldToggle, newToggle) -> {
            if (newToggle != null) {
                RadioButton selectedRadioButton = (RadioButton) newToggle;
                selectionStatus.setText("Selected: " + selectedRadioButton.getText());
                System.out.println("New selection: " + selectedRadioButton.getText());
            } else {
                selectionStatus.setText("No payment method selected.");
                System.out.println("No selection.");
            }
        });


        root.getChildren().addAll(header, creditCard, paypal, bankTransfer, new Label(),
selectionStatus);

        Scene scene = new Scene(root, 400, 300);
        primaryStage.setTitle("RadioButton Demo");
```

```
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

ComboBox

- **Purpose:** Presents a dropdown list of options, allowing the user to select one. Can be editable or non-editable.
- **Key Concept:** ObservableList for holding the items.
- **Common Properties:** items (the ObservableList), value (currently selected item), editable (true/false).
- **Events:** ActionEvent (when selection changes or item is committed in editable mode), valueProperty().addListener() (recommended).

Demo: ComboBox for Country Selection

```
import javafx.application.Application;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.ComboBox;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class ComboBoxDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(15);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.TOP_LEFT);

        Label header = new Label("Select Your Country:");
        header.setStyle("-fx-font-size: 18px; -fx-font-weight: bold;");

        // Data for the ComboBox
        ObservableList<String> countries = FXCollections.observableArrayList(
                "Ethiopia", "Kenya", "Tanzania", "Uganda", "Rwanda", "Sudan", "Somalia", "Eritrea"
        );

        ComboBox<String> countryComboBox = new ComboBox<>();
        countryComboBox.setItems(countries); // Set the data source
        countryComboBox.setPromptText("Choose a country..."); // Placeholder text
        countryComboBox.setEditable(true); // Allow user to type and filter
        countryComboBox.getSelectionModel().selectFirst(); // Select the first item by default

        Label selectionStatus = new Label("Selected Country: Ethiopia");

        // --- Event Handling for ComboBox Selection Change ---
        // 1. Using valueProperty listener (recommended for reacting to selection changes)
        countryComboBox.valueProperty().addListener((observable, oldValue, newValue) -> {
            if (newValue != null) {
                selectionStatus.setText("Selected Country: " + newValue);
                System.out.println("ComboBox selection changed to: " + newValue);
            } else {
                selectionStatus.setText("No country selected.");
            }
        });
```

```
        // 2. Using setOnAction (fires when an item is selected OR Enter is pressed in editable
mode)

        countryComboBox.setOnAction(e -> {
            System.out.println("ComboBox Action (Enter/Selection): " + countryComboBox.getValue());

            if (countryComboBox.isEditable() && countryComboBox.getEditor().getText() != null &&
!countryComboBox.getEditor().getText().equals(countryComboBox.getValue()))
             {
            // This means the user typed something and pressed enter, but it's not in the list or a new value.
                // You might want to add it to the list or handle it as a custom input.
               System.out.println("User entered custom value: " +
                countryComboBox.getEditor().getText());
            }
        });


        root.getChildren().addAll(header, countryComboBox, selectionStatus);

        Scene scene = new Scene(root, 400, 300);
        primaryStage.setTitle("ComboBox Demo");


       primaryStage.setScene(scene);
       primaryStage.show();

    }

    public static void main(String[] args) {
    launch(args);
    }


}
```

ListView

- Purpose: Displays a scrollable list of items, allowing single or multiple selections.
- Key Concept: `ObservableList` for the items.
- Common Properties: `items`, `selectionModel` (for managing selections).
- Events: `selectionModel.selectedItemProperty().addListener()` (for single selection changes), `selectionModel.selectedItemsProperty().addListener()` (for multiple selection changes). `MouseEvent` can also be used for specific clicks.

```
Demo: ListView with Single Selection and Event Handling

import javafx.application.Application;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.ListView;
import javafx.scene.control.SelectionMode; // For single/multiple selection
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class ListViewDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(15);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.TOP_LEFT);

        Label header = new Label("Choose Your Favorite Fruits:");
        header.setStyle("-fx-font-size: 18px; -fx-font-weight: bold;");

        ObservableList<String> fruits = FXCollections.observableArrayList(
                "Apple", "Banana", "Orange", "Grape", "Mango", "Pineapple", "Strawberry",
"Watermelon", "Kiwi"
        );

        ListView<String> fruitListView = new ListView<>(fruits);
        fruitListView.setPrefHeight(150); // Set a preferred height to make it scrollable
        fruitListView.getSelectionModel().setSelectionMode(SelectionMode.SINGLE); // Default is
single

        Label selectionStatus = new Label("Selected: None");

        // --- Event Handling for ListView Selection Change ---
        // Listen to the selectedItemProperty for single selection changes
        fruitListView.getSelectionModel().selectedItemProperty().addListener((observable, oldValue,
newValue) -> {
            if (newValue != null) {
                selectionStatus.setText("Selected: " + newValue);
                System.out.println("ListView selection changed to: " + newValue);
            } else {
                selectionStatus.setText("Selected: None");
                System.out.println("ListView: No item selected.");
            }
        });

        // Demo for multiple selection (uncomment to test)
        /*
        fruitListView.getSelectionModel().setSelectionMode(SelectionMode.MULTIPLE);
        fruitListView.getSelectionModel().selectedItemsProperty().addListener((observable,
oldValues, newValues) -> {
            if (newValues != null && !newValues.isEmpty()) {
                String selected = String.join(", ", newValues);
```

```
                selectionStatus.setText("Selected: " + selected);
                System.out.println("ListView multiple selection: " + selected);
            } else {
                selectionStatus.setText("Selected: None");
            }
        });
        */


        root.getChildren().addAll(header, fruitListView, selectionStatus);

        Scene scene = new Scene(root, 400, 400);
        primaryStage.setTitle("ListView Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

3.4. Composite UI Controls

Composite controls are more complex components that often display structured data or allow sophisticated interactions. They typically involve a combination of basic controls and custom rendering logic.

TableView

- **Purpose:** Displays data in a tabular format (rows and columns). Highly customizable.
- **Key Concepts:**
  - **Model Class:** You'll typically have a POJO (Plain Old Java Object) to represent each row's data (e.g., a Person class, a Product class).
  - **ObservableList<S>:** The data source for the TableView, where S is your model class. This list should be observable so the TableView automatically updates when the data changes.
  - **TableColumn<S, T>:** Represents a single column in the table. S is the type of your model class, and T is the type of data displayed in that column (e.g., String, Integer).
  - **PropertyValueFactory<S, T>:** A convenient way to link a TableColumn to a property of your model class using its getter method name (e.g., "firstName" for a getFirstName() method).
  - **Cell Factories:** For custom rendering of cells (e.g., showing an image, a button, or applying custom styling based on data).
- **Events:** selectionModel.selectedItemProperty().addListener() for row selection.

Demo Example: TableView for Student Data (Populating, Custom Cell Factories, Row Selection)

Let's start by defining a simple Student model class.

```
    // File: Student.java
import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.beans.property.SimpleBooleanProperty;

public class Student {
    private final SimpleStringProperty studentId;
    private final SimpleStringProperty firstName;
    private final SimpleStringProperty lastName;
    private final SimpleIntegerProperty age;
    private final SimpleBooleanProperty enrolled; // New property for custom cell

    public Student(String studentId, String firstName, String lastName, int age, boolean enrolled)
{
        this.studentId = new SimpleStringProperty(studentId);
        this.firstName = new SimpleStringProperty(firstName);
        this.lastName = new SimpleStringProperty(lastName);
        this.age = new SimpleIntegerProperty(age);
        this.enrolled = new SimpleBooleanProperty(enrolled);
    }

    // Getters for properties (essential for PropertyValueFactory)
    public String getStudentId() { return studentId.get(); }
    public String getFirstName() { return firstName.get(); }
    public String getLastName() { return lastName.get(); }
    public int getAge() { return age.get(); }
    public boolean isEnrolled() { return enrolled.get(); } // Note 'is' for boolean

    // Property getters (for more advanced binding, covered later)
    public SimpleStringProperty studentIdProperty() { return studentId; }
    public SimpleStringProperty firstNameProperty() { return firstName; }
    public SimpleStringProperty lastNameProperty() { return lastName; }
    public SimpleIntegerProperty ageProperty() { return age; }
    public SimpleBooleanProperty enrolledProperty() { return enrolled; }

    // Setters (if you want to make the table editable, but often not needed for display)
    public void setStudentId(String studentId) { this.studentId.set(studentId); }
```

```
    public void setFirstName(String firstName) { this.firstName.set(firstName); }
    public void setLastName(String lastName) { this.lastName.set(lastName); }
    public void setAge(int age) { this.age.set(age); }
    public void setEnrolled(boolean enrolled) { this.enrolled.set(enrolled); }
}
```

Now, the TableView demo:

```
import javafx.application.Application;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.TableCell;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.stage.Stage;

// Assuming Student.java is in the same package or correctly imported

public class TableViewDemo extends Application {

    // ObservableList to hold student data for the TableView
    private final ObservableList<Student> studentData = FXCollections.observableArrayList(
            new Student("HU1001", "Abebe", "Kebede", 20, true),
            new Student("HU1002", "Chala", "Lemma", 21, true),
            new Student("HU1003", "Almaz", "Tadesse", 19, false),
            new Student("HU1004", "Fatuma", "Ali", 22, true),
            new Student("HU1005", "Dawit", "Bekele", 20, false)
    );

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(10);
        root.setPadding(new Insets(20));

        Label header = new Label("Hawassa University Students");
        header.setStyle("-fx-font-size: 20px; -fx-font-weight: bold;");

        TableView<Student> studentTable = new TableView<>();
        studentTable.setItems(studentData); // Set the data source

        // --- Define TableColumns ---

        // Student ID Column
        TableColumn<Student, String> idCol = new TableColumn<>("Student ID");
        // Link to the "studentId" property of the Student object (uses getStudentId() internally)
        idCol.setCellValueFactory(new PropertyValueFactory<>("studentId"));
        idCol.setMinWidth(100);

        // First Name Column
        TableColumn<Student, String> firstNameCol = new TableColumn<>("First Name");
        firstNameCol.setCellValueFactory(new PropertyValueFactory<>("firstName"));
        firstNameCol.setMinWidth(120);

        // Last Name Column
        TableColumn<Student, String> lastNameCol = new TableColumn<>("Last Name");
        lastNameCol.setCellValueFactory(new PropertyValueFactory<>("lastName"));
        lastNameCol.setMinWidth(120);

        // Age Column
        TableColumn<Student, Integer> ageCol = new TableColumn<>("Age");
        ageCol.setCellValueFactory(new PropertyValueFactory<>("age"));
```

```
        ageCol.setMinWidth(60);
        ageCol.setStyle("-fx-alignment: CENTER;"); // Center align numbers

        // Enrolled Status Column with Custom Cell Factory
        TableColumn<Student, Boolean> enrolledCol = new TableColumn<>("Enrolled");
        enrolledCol.setCellValueFactory(new PropertyValueFactory<>("enrolled"));
        enrolledCol.setMinWidth(100);
        // Custom Cell Factory: Displays "Yes" in green or "No" in red based on boolean value
        enrolledCol.setCellFactory(column -> {
            return new TableCell<Student, Boolean>() {
                @Override
                protected void updateItem(Boolean item, boolean empty) {
                    super.updateItem(item, empty);
                    if (empty || item == null) {
                        setText(null);
                        setStyle("");
                    } else {
                        setText(item ? "Yes" : "No");
                        if (item) {
                            setTextFill(Color.GREEN);
                        } else {
                            setTextFill(Color.RED);
                        }
                    }
                }
            };
        });

        // Add columns to the table
        studentTable.getColumns().addAll(idCol, firstNameCol, lastNameCol, ageCol, enrolledCol);

        // --- Row Selection Event Handling ---
        Label selectedStudentLabel = new Label("Selected Student: None");
        studentTable.getSelectionModel().selectedItemProperty().addListener((observable,
oldSelection, newSelection) -> {
            if (newSelection != null) {
                selectedStudentLabel.setText(String.format("Selected Student: %s %s (%s)",
                        newSelection.getFirstName(), newSelection.getLastName(),
newSelection.getStudentId()));
                System.out.println("Selected: " + newSelection.getFirstName() + " " +
newSelection.getLastName());
            } else {
                selectedStudentLabel.setText("Selected Student: None");
                System.out.println("No student selected.");
            }
        });

        // Example: Add a new student button
        Button addStudentBtn = new Button("Add New Student");
        addStudentBtn.setOnAction(e -> {
            // In a real application, this would open a dialog to input student details
            // For demo, we just add a predefined one
            studentData.add(new Student("HU1006", "New", "Student", 23, true));
            System.out.println("New student added.");
        });

        root.getChildren().addAll(header, studentTable, selectedStudentLabel, addStudentBtn);

        Scene scene = new Scene(root, 700, 500);
        primaryStage.setTitle("TableView Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

- The Student class is a simple POJO. PropertyValueFactory uses the getter names (e.g., getStudentId()) to retrieve data for each cell.
- idCol.setCellValueFactory(new PropertyValueFactory<>("studentId")) is the core line for linking data to columns.
- enrolledCol.setCellFactory(...) demonstrates how to create a TableCell that customizes the visual representation of the Boolean enrolled property, showing "Yes" in green or "No" in red.
- studentTable.getSelectionModel().selectedItemProperty().addListener(...) is used to react when a user selects a row, updating the selectedStudentLabel.

TreeView

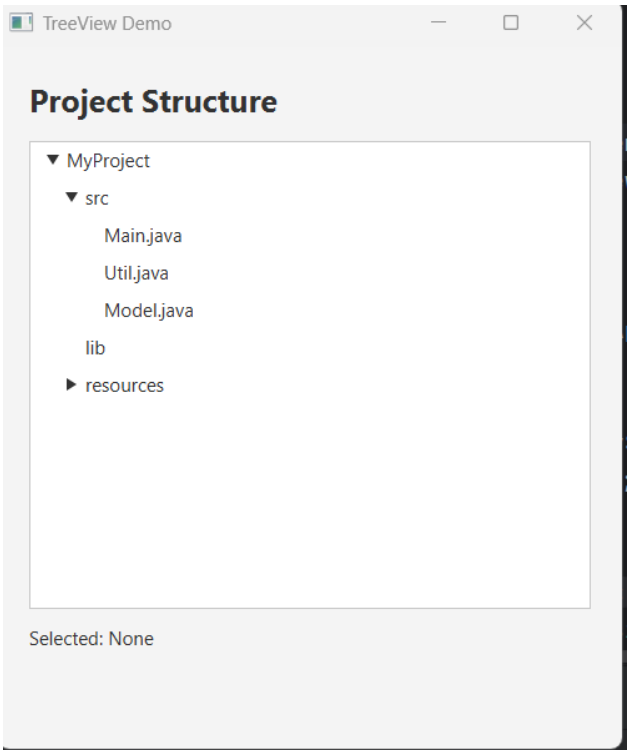
- **Purpose:** Displays hierarchical data in a tree-like structure.
- **Key Concept:** TreeItem<T>. Each node in the tree is a TreeItem, where T is the type of data it holds. TreeItems can have children TreeItems.
- **Common Properties:** root (the top-most TreeItem), showRoot, selectionModel.
- **Events:** selectionModel.selectedItemProperty().addListener() for item selection.

Demo Example: TreeView for File System-like Structure

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.TreeItem;
import javafx.scene.control.TreeView;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class TreeViewDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(10);
        root.setPadding(new Insets(20));

        Label header = new Label("Project Structure");
        header.setStyle("-fx-font-size: 20px; -fx-font-weight: bold;");

        // Create the root TreeItem (can be hidden or shown)
        TreeItem<String> rootItem = new TreeItem<>("MyProject");
        rootItem.setExpanded(true); // Expand the root by default

        // Create branches (directories)
        TreeItem<String> srcFolder = new TreeItem<>("src");
        srcFolder.setExpanded(true);
        TreeItem<String> libFolder = new TreeItem<>("lib");
        TreeItem<String> resFolder = new TreeItem<>("resources");

        // Add files to src
        srcFolder.getChildren().addAll(
                new TreeItem<>("Main.java"),
                new TreeItem<>("Util.java"),
                new TreeItem<>("Model.java")
        );

        // Add files to resources
        resFolder.getChildren().addAll(
                new TreeItem<>("config.properties"),
                new TreeItem<>("images/") // Subfolder, though just a label here
        );

        // Add branches to the root
        rootItem.getChildren().addAll(srcFolder, libFolder, resFolder);

        // Create the TreeView with the root item
        TreeView<String> treeView = new TreeView<>(rootItem);
        treeView.setShowRoot(true); // Show the "MyProject" root item
        treeView.setPrefHeight(300);

        Label selectedItemLabel = new Label("Selected: None");

        // --- Event Handling for TreeView Selection ---
        treeView.getSelectionModel().selectedItemProperty().addListener((observable, oldItem,
newItem) -> {
            if (newItem != null) {
                selectedItemLabel.setText("Selected: " + newItem.getValue());
                System.out.println("TreeView selection changed to: " + newItem.getValue());
```

```
            } else {
                selectedItemLabel.setText("Selected: None");
                System.out.println("TreeView: No item selected.");
            }
        });

        root.getChildren().addAll(header, treeView, selectedItemLabel);

        Scene scene = new Scene(root, 400, 450);
        primaryStage.setTitle("TreeView Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

- The TreeItem<String> holds a String value (the folder/file name). You can use any object type for T.
- rootItem.setExpanded(true) makes the item initially open.
- treeView.setShowRoot(true) determines if the very top rootItem itself is visible.
- The selectionModel listener is similar to ListView's.



ProgressBar

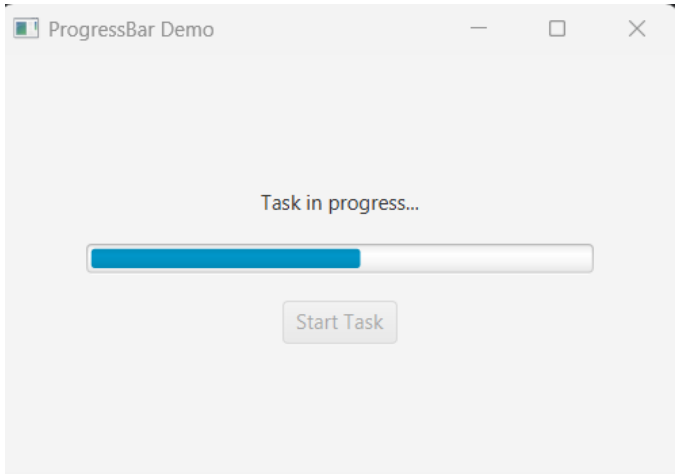
- **Purpose:** Shows the progress of a task, usually a background operation.
- **Common Properties:** progress (a double from 0.0 to 1.0, or ProgressBar.INDETERMINATE_PROGRESS for indefinite progress).
- **Events:** progressProperty().addListener() to react to changes.

Demo: ProgressBar with Background Task Simulation

```
import javafx.application.Application;
import javafx.application.Platform;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.control.ProgressBar;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class ProgressBarDemo extends Application {

    private ProgressBar progressBar;
    private Label statusLabel;
    private Button startButton;

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(15);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.CENTER);

        statusLabel = new Label("Click 'Start Task' to begin.");
        progressBar = new ProgressBar(0); // Initial progress 0
        progressBar.setPrefWidth(300);

        startButton = new Button("Start Task");
        startButton.setOnAction(e -> startBackgroundTask());

        root.getChildren().addAll(statusLabel, progressBar, startButton);

        Scene scene = new Scene(root, 400, 250);
        primaryStage.setTitle("ProgressBar Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    private void startBackgroundTask() {
        startButton.setDisable(true);
        progressBar.setProgress(0); // Reset progress
        statusLabel.setText("Task in progress...");

        // Simulate a background task using a new Thread
        Thread backgroundThread = new Thread(() -> {
            try {
                for (int i = 0; i <= 100; i += 5) {
                    final double progress = (double) i / 100;
                    // Update UI on the JavaFX Application Thread
                    Platform.runLater(() -> progressBar.setProgress(progress));
                    Thread.sleep(200); // Simulate work
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            } finally {
                Platform.runLater(() -> {
                    statusLabel.setText("Task complete!");
                    progressBar.setProgress(1.0); // Ensure it's 100%
```

```
                    startButton.setDisable(false);
                });
            }
        });
        backgroundThread.setDaemon(true); // Allow application to exit if thread is still running
        backgroundThread.start();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

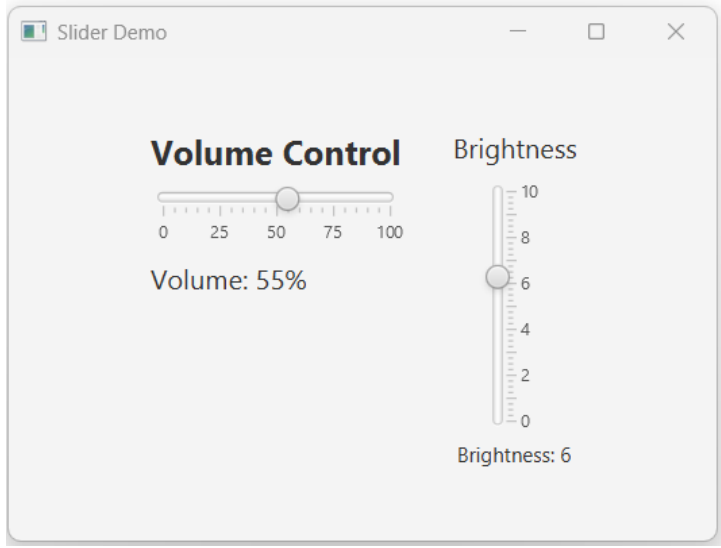
- ProgressBar.setProgress(double value) updates the visual progress.
- Platform.runLater(() -> ...) is crucial: UI updates *must* happen on the JavaFX Application Thread. You cannot directly update UI controls from a background thread.
- For indefinite progress, set progressBar.setProgress(ProgressBar.INDETERMINATE_PROGRESS).



Slider

- **Purpose:** Allows users to select a value from a continuous or discrete range by dragging a "thumb."
- **Common Properties:** min, max, value (current position of thumb), majorTickUnit, minorTickCount, showTickLabels, showTickMarks, snapToTicks, orientation.
- **Events:** valueProperty().addListener() for real-time value changes.

Demo: Slider for Volume Control



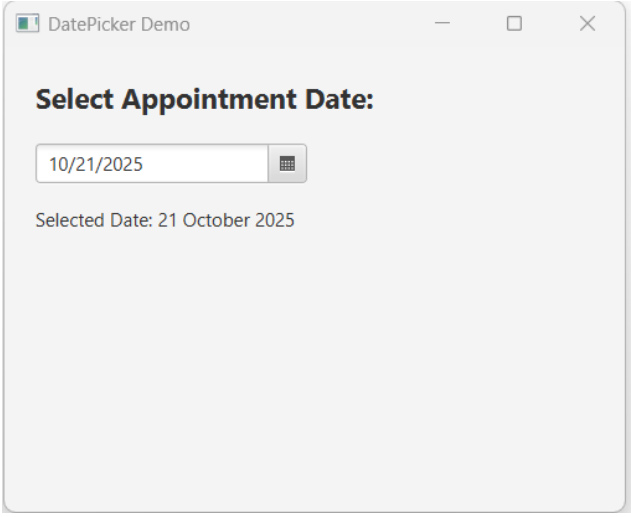

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Orientation;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.Slider;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class SliderDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
```

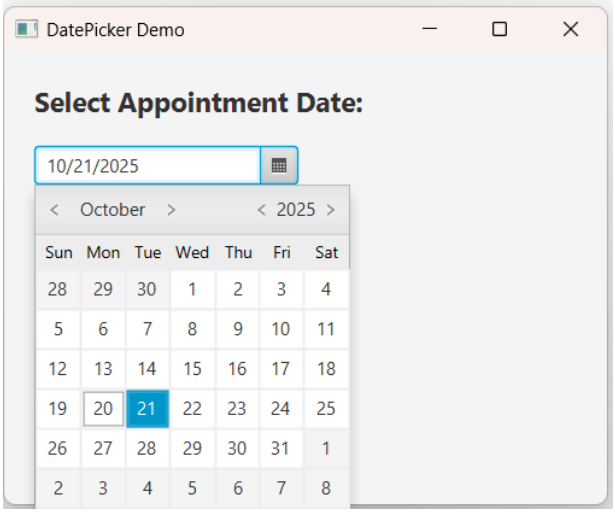
```
        VBox root = new VBox(20);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.CENTER);

        Label header = new Label("Volume Control");
        header.setStyle("-fx-font-size: 20px; -fx-font-weight: bold;");

        // Horizontal Slider for Volume
        Slider volumeSlider = new Slider(0, 100, 50); // Min=0, Max=100, Initial=50
        volumeSlider.setMajorTickUnit(25); // Show ticks every 25 units
        volumeSlider.setMinorTickCount(4); // 4 minor ticks between major ticks (5 divisions)
        volumeSlider.setShowTickLabels(true); // Show numeric labels
        volumeSlider.setShowTickMarks(true); // Show tick marks
        volumeSlider.setSnapToTicks(true); // Snap to tick marks when dragged
        volumeSlider.setBlockIncrement(10); // How much value changes on click on track

        Label volumeLabel = new Label("Volume: 50%");
        volumeLabel.setStyle("-fx-font-size: 16px;");

        // --- Event Handling for Slider Value Change ---
        volumeSlider.valueProperty().addListener((observable, oldValue, newValue) -> {
            // Round to nearest integer for display
            int currentVolume = newValue.intValue();
            volumeLabel.setText("Volume: " + currentVolume + "%");
            System.out.println("Volume changed to: " + currentVolume);
            // In a real app, you'd adjust actual audio volume here
        });

        // Vertical Slider for Brightness (another example)
        Label brightnessHeader = new Label("Brightness");
        brightnessHeader.setStyle("-fx-font-size: 16px;");

        Slider brightnessSlider = new Slider(0, 10, 5);
        brightnessSlider.setOrientation(Orientation.VERTICAL);
        brightnessSlider.setMajorTickUnit(1);
        brightnessSlider.setShowTickLabels(true);
        brightnessSlider.setShowTickMarks(true);
        brightnessSlider.setSnapToTicks(true);
        brightnessSlider.setPrefHeight(150); // Give it some height

        Label brightnessLabel = new Label("Brightness: 5");
        brightnessSlider.valueProperty().addListener((obs, oldVal, newVal) -> {
            brightnessLabel.setText("Brightness: " + newVal.intValue());
        });

        HBox slidersBox = new HBox(30);
        slidersBox.setAlignment(Pos.CENTER);
        VBox brightnessVBox = new VBox(5, brightnessHeader, brightnessSlider, brightnessLabel);

         brightnessVBox.setAlignment(Pos.CENTER);
         slidersBox.getChildren().addAll(new VBox(5, header, volumeSlider, volumeLabel),
         brightnessVBox);


    root.getChildren().addAll(slidersBox);

    Scene scene = new Scene(root, 500, 400);
    primaryStage.setTitle("Slider Demo");
    primaryStage.setScene(scene);
    primaryStage.show();
}

public static void main(String[] args) {
    launch(args);
}


}
```

DatePicker

- **Purpose**:Provides a calendar-based control for selecting a date.
- **Common Properties**: `value` (a `LocalDate` object), `converter` (for custom date formatting), `dayCellFactory` (for custom styling of calendar cells).
- **Events**: `valueProperty().addListener()` to react to date selection changes. `setOnAction` also works.

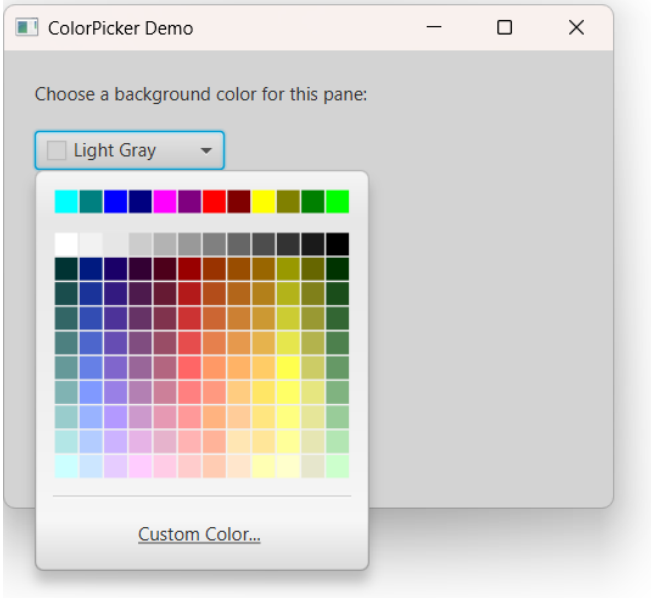
```
**Demo: DatePicker for an Appointment**

import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.DatePicker;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class DatePickerDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(15);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.TOP_LEFT);

        Label header = new Label("Select Appointment Date:");
        header.setStyle("-fx-font-size: 18px; -fx-font-weight: bold;");

        DatePicker appointmentDatePicker = new DatePicker();
        appointmentDatePicker.setPromptText("Click to select a date");

        // Set an initial date
        appointmentDatePicker.setValue(LocalDate.now().plusDays(1)); // Tomorrow's date

        Label selectedDateLabel = new Label("Selected Date: " +
            appointmentDatePicker.getValue().format(DateTimeFormatter.ofPattern("dd MMMM yyyy")));

        // --- Event Handling for DatePicker Value Change ---
        appointmentDatePicker.valueProperty().addListener((observable, oldValue, newValue) -> {
            if (newValue != null) {
                DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd MMMM yyyy");
                selectedDateLabel.setText("Selected Date: " + newValue.format(formatter));
                System.out.println("New date selected: " + newValue);
            } else {
                selectedDateLabel.setText("Selected Date: None");
            }
        });

        root.getChildren().addAll(header, appointmentDatePicker, selectedDateLabel);

        Scene scene = new Scene(root, 400, 300);
        primaryStage.setTitle("DatePicker Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```


DatePicker Demo
Select Appointment Date:
10/21/2025
Selected Date: 21 October 2025


DatePicker Demo
Select Appointment Date:
10/21/2025
October
2025
Sun Mon Tue Wed Thu Fri Sat
28 29 30 1 2 3 4
5 6 7 8 9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31 1
2 3 4 5 6 7 8

ColorPicker

- **Purpose:** Allows users to select a color from a predefined palette or a custom color chooser.
- **Common Properties:** value (a Color object).
- **Events:** valueProperty().addListener() or setOnAction.



Demo: ColorPicker for Background Color

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.ColorPicker;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.scene.paint.Color;
import javafx.stage.Stage;

public class ColorPickerDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(15);
        root.setPadding(new Insets(20));
        root.setStyle("-fx-font-size: 16px;");

        Label header = new Label("Choose a background color for this pane:");
        ColorPicker colorPicker = new ColorPicker();
        colorPicker.setValue(Color.LIGHTGRAY); // Initial color

        // --- Event Handling for ColorPicker Value Change ---
        colorPicker.setOnAction(e -> {
            Color selectedColor = colorPicker.getValue();
            String hexColor = toHexString(selectedColor);
            root.setStyle("-fx-background-color: " + hexColor + ";");
            System.out.println("Color changed to: " + hexColor);
        });

        // Set the initial background color
        root.setStyle("-fx-background-color: " + toHexString(colorPicker.getValue()) + ";");

        root.getChildren().addAll(header, colorPicker);

        Scene scene = new Scene(root, 400, 300);
        primaryStage.setTitle("ColorPicker Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    // Helper method to convert a JavaFX Color object to a CSS-compatible hex string
    private String toHexString(Color color) {
        return String.format("#%02X%02X%02X",
                (int) (color.getRed() * 255),
                (int) (color.getGreen() * 255),
                (int) (color.getBlue() * 255));
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Building a Custom Modal Dialog (Lab Practice-2 compatible)

A modal dialog is a window that blocks interaction with the main application until it is closed. JavaFX has a built-in Dialog framework, but creating one from scratch using a Stage is an excellent learning exercise.

Key Concepts:

- Create a new Stage.
- Initialize its modality to Modality.APPLICATION_MODAL.
- Set its owner to the primary Stage.
- Build the UI for the dialog.
- Use stage.showAndWait() to block the main application.
- Create a mechanism to return a value from the dialog.

Demo: Custom "Confirm" Dialog that Returns a Value

First, let's create a reusable ConfirmBox class.

```
    // File: ConfirmBox.java
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.stage.Modality;
import javafx.stage.Stage;

public class ConfirmBox {

    private static boolean answer;

    public static boolean display(String title, String message) {
        Stage window = new Stage();

        // --- Window Setup ---
        // Block events to other windows until this one is closed
        window.initModality(Modality.APPLICATION_MODAL);
        window.setTitle(title);
        window.setMinWidth(250);

        // --- UI Controls ---
        Label label = new Label(message);

        Button yesButton = new Button("Yes");
        yesButton.setOnAction(e -> {
            answer = true;
            window.close();
        });

        Button noButton = new Button("No");
        noButton.setOnAction(e -> {
            answer = false;
            window.close();
        });

        // --- Layout ---
        HBox buttonLayout = new HBox(10);
        buttonLayout.getChildren().addAll(yesButton, noButton);
        buttonLayout.setAlignment(Pos.CENTER);

        VBox layout = new VBox(20);
        layout.getChildren().addAll(label, buttonLayout);
        layout.setAlignment(Pos.CENTER);
```

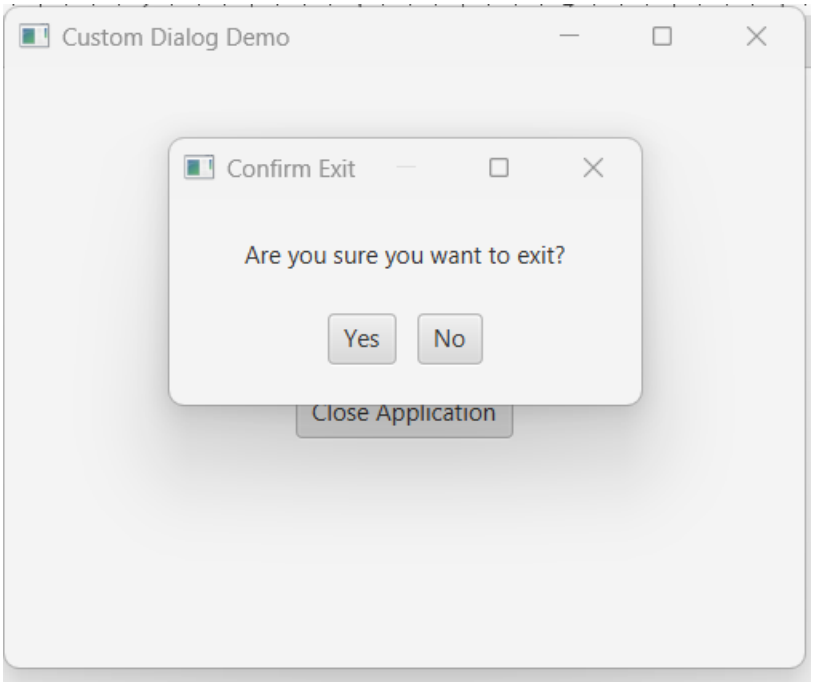
```
        layout.setPadding(new Insets(20));

        // --- Scene ---
        Scene scene = new Scene(layout);
        window.setScene(scene);

        // --- Show and Wait ---
        // showAndWait() blocks execution until the window is closed
        window.showAndWait();

        return answer;
    }
}
```

Now, let's use this ConfirmBox in a main application.

```
    // File: CustomDialogDemo.java
import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class CustomDialogDemo extends Application {

    private Stage primaryStage;
    private Label statusLabel;

    @Override
    public void start(Stage primaryStage) {
        this.primaryStage = primaryStage; // Store the primary stage for ownership
        primaryStage.setTitle("Custom Dialog Demo");

        statusLabel = new Label("Click a button to open a dialog.");
        statusLabel.setStyle("-fx-font-size: 16px;");

        Button closeButton = new Button("Close Application");
        closeButton.setOnAction(e -> {
            boolean result = ConfirmBox.display("Confirm Exit", "Are you sure you want to exit?");
            if (result) {
                System.out.println("User chose to exit.");
                primaryStage.close();
            } else {
                System.out.println("User chose NOT to exit.");
                statusLabel.setText("Exit was cancelled.");
            }
        });

        VBox layout = new VBox(20);
        layout.setAlignment(Pos.CENTER);
        layout.getChildren().addAll(statusLabel, closeButton);

        Scene scene = new Scene(layout, 400, 300);
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

- The ConfirmBox class is a self-contained, reusable component.
- window.initModality(Modality.APPLICATION_MODAL) is the key to making the dialog modal.

- window.showAndWait() pauses the execution of the calling method until the ConfirmBox window is closed.
- The static boolean answer is used to store and return the user's choice.



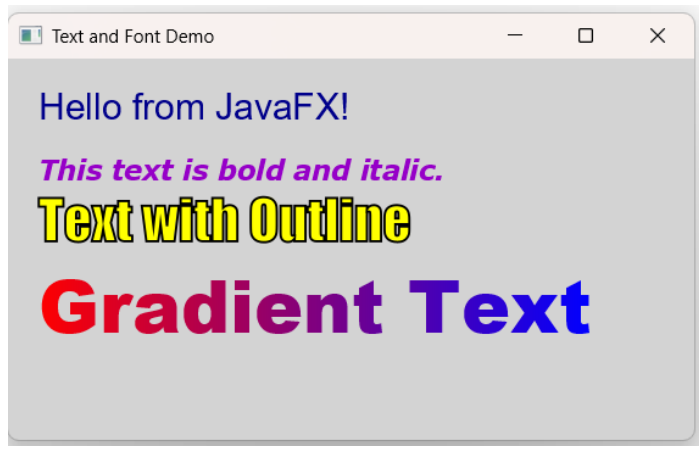
3.5. Shapes, Colors, Texts, Fonts, Lines, Circles, Rectangles, CSS Styling

JavaFX provides a rich set of APIs for drawing 2D shapes and styling them with colors, fonts, and CSS. All shapes are Nodes and can be added directly to the Scene Graph.

3.5.1. Color, Texts, Fonts

- **Color Class:** Represents colors in the RGBA (Red, Green, Blue, Alpha/Opacity) color space.
  - **Specifying Colors:**
    - **Named Colors:** Color.RED, Color.BLUE, Color.LIGHTGRAY.
    - **RGB:** Color.rgb(255, 0, 0) for red. Values are 0-255.
    - **RGB with Opacity:** Color.rgb(0, 0, 255, 0.5) for semi-transparent blue. Opacity is 0.0 (transparent) to 1.0 (opaque).
    - **Grayscale:** Color.gray(0.5) for 50% gray.
    - **Hex:** Color.web("#FF0000") for red or Color.web("#0000FF80") for semi-transparent blue.
- **Text Node:** A Node for displaying a string of text.
  - **Properties:** text, x, y (position), fill (color), stroke (outline color), font.
- **Font Class:** Used to define the font for text.
  - **Specifying Fonts:**
    - **Constructor:** new Font("Arial", 24).
    - **Static Methods:** Font.font("Verdana", FontWeight.BOLD, FontPosture.ITALIC, 20). FontWeight can be BOLD, NORMAL, etc. FontPosture can be ITALIC, REGULAR.

Demo: Displaying Styled Text




```
import javafx.application.Application;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.FontPosture;
import javafx.scene.text.FontWeight;
import javafx.scene.text.Text;
import javafx.stage.Stage;

public class TextAndFontDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        // A Group is a simple container that doesn't apply any layout policy
        Group root = new Group();

        // Simple Text
        Text t1 = new Text(20, 40, "Hello from JavaFX!");
        t1.setFont(new Font("Arial", 24));
        t1.setFill(Color.DARKBLUE);

        // Styled Text
        Text t2 = new Text(20, 80, "This text is bold and italic.");
        t2.setFont(Font.font("Verdana", FontWeight.BOLD, FontPosture.ITALIC, 18));
        t2.setFill(Color.rgb(150, 0, 200)); // Purple color

        // Text with Stroke (outline)
        Text t3 = new Text(20, 120, "Text with Outline");
        t3.setFont(Font.font("Impact", FontWeight.NORMAL, 36));
        t3.setFill(Color.YELLOW);
        t3.setStroke(Color.BLACK); // Black outline
        t3.setStrokeWidth(1.5);

        // Text with Gradient Fill
        Text t4 = new Text(20, 180, "Gradient Text");
        t4.setFont(Font.font("Arial Black", 48));
        javafx.scene.paint.LinearGradient gradient = new javafx.scene.paint.LinearGradient(
```

```
                0, 0, 1, 0, true, javafx.scene.paint.CycleMethod.NO_CYCLE,
                new javafx.scene.paint.Stop(0, Color.RED),
                new javafx.scene.paint.Stop(1, Color.BLUE)
        );
        t4.setFill(gradient);


        root.getChildren().addAll(t1, t2, t3, t4);

        Scene scene = new Scene(root, 450, 250, Color.LIGHTGRAY);
        primaryStage.setTitle("Text and Font Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

3.5.2. Lines, Circle, Rectangle (and other basic shapes)

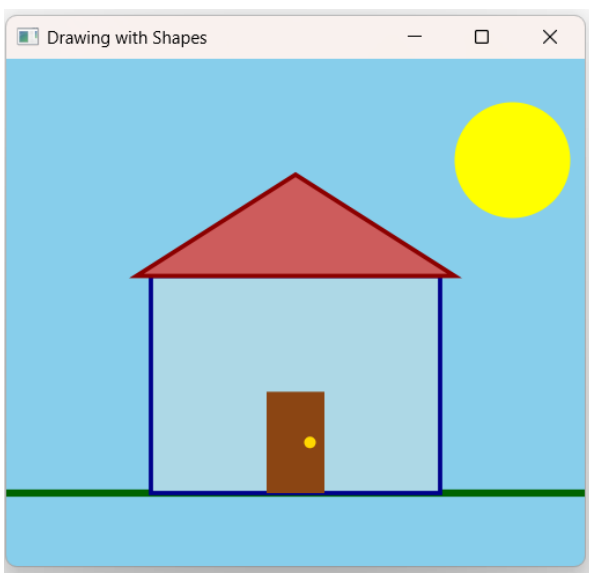
All shape classes are in the javafx.scene.shape package.

- **Common Properties:** fill (the interior color/paint), stroke (the outline color/paint), strokeWidth.
- **Line:** Defined by startX, startY, endX, endY.
- **Rectangle:** Defined by x, y, width, height. Can have rounded corners with arcWidth, arcHeight.
- **Circle:** Defined by centerX, centerY, radius.
- **Ellipse:** Defined by centerX, centerY, radiusX, radiusY.
- **Polygon:** A closed shape defined by a series of x,y coordinate pairs.
- **Polyline:** An open series of connected lines defined by x,y coordinate pairs.
- **Arc:** A portion of an ellipse, defined by center, radii, startAngle, length (angular extent), and ArcType (OPEN, CHORD, ROUND).

Demo: Drawing a Simple Composition

```
import javafx.application.Application;
import javafx.scene.Group;
import javafx.scene.Scene;
import javafx.scene.paint.Color;
import javafx.scene.shape.Circle;
import javafx.scene.shape.Line;
import javafx.scene.shape.Polygon;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;

public class ShapesDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        Group root = new Group();

        // A simple house
        // The main body (rectangle)
        Rectangle houseBody = new Rectangle(100, 150, 200, 150);
        houseBody.setFill(Color.LIGHTBLUE);
        houseBody.setStroke(Color.DARKBLUE);
        houseBody.setStrokeWidth(3);

        // The roof (polygon)
        Polygon roof = new Polygon();
        roof.getPoints().addAll(new Double[]{
            90.0, 150.0,  // Top-left point
            200.0, 80.0,  // Peak
```

```
            310.0, 150.0  // Top-right point
        });
        roof.setFill(Color.INDIANRED);
        roof.setStroke(Color.DARKRED);
        roof.setStrokeWidth(3);

        // The door (rectangle)
        Rectangle door = new Rectangle(180, 230, 40, 70);
        door.setFill(Color.SADDLEBROWN);

        // The doorknob (circle)
        Circle doorknob = new Circle(210, 265, 4);
        doorknob.setFill(Color.GOLD);

        // The sun (circle)
        Circle sun = new Circle(350, 70, 40);
        sun.setFill(Color.YELLOW);

        // Ground line
        Line ground = new Line(0, 300, 400, 300);
        ground.setStroke(Color.DARKGREEN);
        ground.setStrokeWidth(5);


        root.getChildren().addAll(ground, houseBody, roof, door, doorknob, sun);

        Scene scene = new Scene(root, 400, 350, Color.SKYBLUE);
        primaryStage.setTitle("Drawing with Shapes");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```



3.5.3. CSS Styling (MAJOR SECTION)

While you can set properties like fill, stroke, and font directly in code, JavaFX's integration with Cascading Style Sheets (CSS) provides a much more powerful and flexible way to style your applications. It promotes the separation of design from logic.

The Power and Importance of CSS in JavaFX

- **Separation of Concerns:** Keep your Java code focused on application logic (what the app *does*) and your CSS focused on appearance (how the app *looks*).
- **Reusability:** Define a style once in a CSS file and apply it to many controls.
- **Ease of Theming:** Change the entire look and feel of your application by simply switching stylesheets.
- **Designer-Friendly:** UI/UX designers who know web CSS can contribute to the look of a JavaFX application without needing to write Java code.
- **Advanced Styling:** CSS allows for complex styling, including pseudo-classes (like :hover, :pressed), gradients, and effects that are more verbose to implement in code.

Selector Types

You can target nodes for styling using selectors, similar to web CSS.

- **Type Selector (or Class Selector):** Targets all instances of a specific JavaFX class. The class name is prefixed with ..

```
.button {
    -fx-font-size: 14px;
    -fx-background-color: #4CAF50; /* Green */
    -fx-text-fill: white;
}
.label {
    -fx-font-weight: bold;
}
```

- **ID Selector:** Targets a single, specific node. You set the ID in your Java code (node.setId("myButton")) and reference it in CSS with #.
  - *Java Code:* Button saveButton = new Button("Save"); saveButton.setId("save-button");
  - *CSS:*

```
#save-button {
    -fx-background-color: #008CBA; /* Blue */
}
```

- **Style Class Selector:** The most common and reusable way. You can add one or more style classes to a node (node.getStyleClass().add("my-class")) and reference it in CSS with ..
  - *Java Code:* Button deleteButton = new Button("Delete"); deleteButton.getStyleClass().add("danger-button");
  - *CSS:*

```
.danger-button {
    -fx-background-color: #f44336; /* Red */
}
```

- **Pseudo-class Selector:** Targets a node in a specific state. These are appended to other selectors with a colon (:).
  - :hover: When the mouse is over the node.
  - :pressed: While the node is being clicked.
  - :focused: When the node has keyboard focus.
  - :disabled: When the node is disabled.
  - :selected: For items in controls like ListView or CheckBox.

```
.button:hover {
    -fx-background-color: #45a049; /* Darker green on hover */
}
.text-field:focused {
    -fx-border-color: cornflowerblue;
    -fx-border-width: 2px;
}
```

- **Descendant Selector:** Targets a node that is a descendant of another node.

```
/* Style only Labels that are inside a VBox with the style class 'profile-section' */
.profile-section .label {
    -fx-text-fill: #555555;
}
```

Note: All JavaFX CSS properties start with **-fx-**

Internal, External, and Inline CSS

1. **Inline CSS:**
    - **How:** Set directly on a node using node.setStyle("-fx-property: value; ...").
    - **Use Case:** Quick, one-off styling for a single node. Good for prototyping or dynamic styling based on application state.
    - **Precedence:** Highest. Inline styles override styles from stylesheets.
    - **Example:** label.setStyle("-fx-background-color: yellow; -fx-padding: 5px;");
2. **Internal CSS (Scene/Parent-level):**
    - **How:** Not directly supported in the same way as <style> tags in HTML. The closest equivalent is defining styles in a string and applying them to a parent node or the scene. This is generally not recommended.
3. **External CSS:**
    - **How:** Create a separate .css file. Load it and apply it to a Scene or a specific parent Node. This is the **recommended approach**.
    - **Use Case:** The standard way to style an entire application or a major component. Promotes separation and reusability.
    - **Precedence:** Lowest. Styles can be overridden by ID selectors, class selectors with more specificity, or inline styles.
    - **Example:**
        - Create styles.css in your project's src folder.
        - Load and apply it in Java:

```
Scene scene = new Scene(root, 600, 400);
scene.getStylesheets().add(getClass().getResource("styles.css").toExternalForm(
));
primaryStage.setScene(scene);
```

Detailed Example: Styling a Custom Dialog with External CSS

Let's restyle the CustomDialogDemo and ConfirmBox from the previous section.

Step 1: Create the CSS file (dialog-styles.css)
Create this file in the same directory as your Java source files.

```
/* File: dialog-styles.css */

/* Style for the root pane of the dialog */
.dialog-pane {
    -fx-background-color: #f4f4f4;
    -fx-border-color: #cccccc;
    -fx-border-width: 1px;
    -fx-effect: dropshadow(three-pass-box, rgba(0,0,0,0.2), 10, 0, 0, 5);
}

/* Style for the message label in the dialog */
.dialog-label {
    -fx-font-family: "Arial";
    -fx-font-size: 16px;
    -fx-text-fill: #333333;
}

/* Base style for all buttons in the dialog */
.dialog-button {
    -fx-background-color: #e0e0e0;
    -fx-text-fill: #222222;
    -fx-font-weight: bold;
    -fx-padding: 8px 16px;
    -fx-background-radius: 5px;
    -fx-border-radius: 5px;
    -fx-border-color: #bbbbbb;
    -fx-border-width: 1px;
}

/* Hover effect for dialog buttons */
.dialog-button:hover {
    -fx-background-color: #d0d0d0;
}

/* Specific style for the 'Yes' or 'Confirm' button */
.confirm-button {
    -fx-background-color: #4CAF50; /* Green */
    -fx-text-fill: white;
    -fx-border-color: #388E3C;
}
.confirm-button:hover {
    -fx-background-color: #45a049;
}

/* Specific style for the 'No' or 'Cancel' button */
.cancel-button {
    -fx-background-color: #f44336; /* Red */
    -fx-text-fill: white;
    -fx-border-color: #d32f2f;
}
.cancel-button:hover {
    -fx-background-color: #e53935;
}
```

Step 2: Modify ConfirmBox.java to use the CSS

```
// File: ConfirmBox.java (Modified)
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.HBox;
import javafx.scene.layout.VBox;
import javafx.stage.Modality;
import javafx.stage.Stage;

public class ConfirmBox {

    private static boolean answer;

    public static boolean display(String title, String message) {
        Stage window = new Stage();
        window.initModality(Modality.APPLICATION_MODAL);
        window.setTitle(title);
        window.setMinWidth(300);

        Label label = new Label(message);
        label.getStyleClass().add("dialog-label"); // Add style class

        Button yesButton = new Button("Yes");
        // Add multiple style classes
        yesButton.getStyleClass().addAll("dialog-button", "confirm-button");
        yesButton.setOnAction(e -> {
            answer = true;
            window.close();
        });

        Button noButton = new Button("No");
        noButton.getStyleClass().addAll("dialog-button", "cancel-button");
        noButton.setOnAction(e -> {
            answer = false;
            window.close();
        });

        HBox buttonLayout = new HBox(15); // Increased spacing
        buttonLayout.getChildren().addAll(yesButton, noButton);
        buttonLayout.setAlignment(Pos.CENTER);

        VBox layout = new VBox(20);
        layout.getChildren().addAll(label, buttonLayout);
        layout.setAlignment(Pos.CENTER);
        layout.setPadding(new Insets(25));
        layout.getStyleClass().add("dialog-pane"); // Add style class to the root pane

        Scene scene = new Scene(layout);
        // --- Apply the external stylesheet ---
        scene.getStylesheets().add(ConfirmBox.class.getResource("dialog-
styles.css").toExternalForm());
        window.setScene(scene);

        window.showAndWait();

        return answer;
    }
}
```

Changes Made:

1. **Added Style Classes:** We used getStyleClass().add(...) to apply the CSS classes we defined (e.g., dialog-pane, dialog-label, confirm-button).

2. **Loaded Stylesheet:** The crucial line scene.getStylesheets().add(...) loads our dialog-styles.css file and applies its rules to the entire scene for the dialog. ConfirmBox.class.getResource(...) is a reliable way to find the CSS file relative to the ConfirmBox.java class file.

Now, when you run CustomDialogDemo, the confirmation dialog will have the new, professional styling from the CSS file.

CSS Styling Best Practices:

- **Prefer External Stylesheets:** Keep CSS separate from Java code for maintainability.
- **Use Style Classes over IDs:** Style classes are reusable. Use IDs only for truly unique elements or for specific lookups in code.
- **Be Specific but Not Overly Specific:** .my-view .button is more specific than .button. This can be useful, but overly long selectors can make CSS hard to override and debug.
- **Organize Your CSS:** Group related styles together (e.g., all button styles, all text field styles). Use comments to document your sections.
- **Default Stylesheet:** JavaFX has a default stylesheet called modena.css that provides the standard look and feel. Your custom CSS overrides these default styles.

3.6. Properties and Bindings (MAJOR SECTION)

Properties and Bindings are among the most powerful features of JavaFX. They form the basis of a reactive programming model that can dramatically simplify your code and reduce bugs, especially when synchronizing data between your application logic (model) and your UI (view).

Fundamental Concept: JavaFX Properties as Observable Values

In standard Java, you use simple fields with getters and setters:

```
private String name;
public String getName() { ... }
public void setName(String name) { ... }
```

In JavaFX, you use **Property objects**:

```
private StringProperty name = new SimpleStringProperty();
public String getName() { return name.get(); }
public void setName(String name) { this.name.set(name); }
public StringProperty nameProperty() { return name; }
```

What's the difference?
A StringProperty (and other property types like **IntegerProperty**, **BooleanProperty**, **ObjectProperty**, **DoubleProperty**) is not just a wrapper for a value. It is an **observable value**. This means ***you can attach listeners to it that are automatically notified whenever its value changes.***

Why use them?

- **Automatic UI Updates:** UI controls in JavaFX are built using properties. For example, TextField has a textProperty(). When you *bind* this property to a property in your data model, the text field will automatically update its display whenever the data model's property changes, and vice-versa, without you writing any manual update code.
- **Simpler Synchronization:** It eliminates a lot of "glue" code. You no longer need to write event handlers just to update a label when a text field changes. You simply bind them together.
- **Enables Reactive Programming:** You can define relationships and dependencies between properties, creating a chain of automatic updates.

Unidirectional Binding (bind())

A unidirectional binding links a target property to a source property. The target property becomes read-only and its value will always be the same as the source property. If the source property's value changes, the target property's value is automatically updated.

Demo: Binding a Label to a Slider's Value

This is a classic example. Instead of using a listener like we did in the Slider demo, we can use a binding.

```java
import javafx.application.Application;
import javafx.beans.binding.Bindings;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.Slider;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class UnidirectionalBindingDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(20);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.CENTER);

        Slider slider = new Slider(0, 100, 25);
        slider.setShowTickLabels(true);
        slider.setShowTickMarks(true);
        slider.setMajorTickUnit(25);

        Label valueLabel = new Label();
        valueLabel.setStyle("-fx-font-size: 20px;");

        // --- Unidirectional Binding ---
        // Bind the textProperty of the label to the valueProperty of the slider.
        // We use Bindings.format to create a formatted string.
        valueLabel.textProperty().bind(
            Bindings.format("Value: %.0f", slider.valueProperty())
        );
        // The label's text will now AUTOMATICALLY update whenever the slider's value changes.
        // NO listener is needed!

        // Another example: bind the disable property of a button to a condition
        javafx.scene.control.Button actionButton = new javafx.scene.control.Button("Action (Enabled
when value > 50)");
        actionButton.disableProperty().bind(slider.valueProperty().lessThanOrEqualTo(50));


        root.getChildren().addAll(slider, valueLabel, actionButton);

        Scene scene = new Scene(root, 400, 300);
        primaryStage.setTitle("Unidirectional Binding Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

- valueLabel.textProperty().bind(...): The textProperty of the label is now bound to a formatted string derived from the slider's valueProperty.
- actionButton.disableProperty().bind(...): The disable property of the button is bound to the result of a boolean expression. The button will automatically become enabled/disabled as you move the slider past 50.

Bidirectional Binding (bindBidirectional())

A bidirectional binding links two properties together. When the value of one property changes, the other is automatically updated, and vice-versa. This is incredibly useful for synchronizing UI controls with data model properties.

Demo: Two TextFields Staying in Sync

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class BidirectionalBindingDemo extends Application {

    @Override
    public void start(Stage primaryStage) {
        VBox root = new VBox(15);
        root.setPadding(new Insets(20));
        root.setAlignment(Pos.CENTER);

        Label label1 = new Label("Field 1:");
        TextField textField1 = new TextField("Initial Text");

        Label label2 = new Label("Field 2 (in sync with Field 1):");
        TextField textField2 = new TextField();

        // --- Bidirectional Binding ---
        // Link the text properties of the two text fields.
        textField1.textProperty().bindBidirectional(textField2.textProperty());
        // Now, typing in either text field will instantly update the other.

        // --- Model-View Sync Example ---
        Label modelLabel = new Label("\n--- Data Model Sync ---");
        modelLabel.setStyle("-fx-font-weight: bold;");

        // Our data model object
        class User {
            private StringProperty username = new SimpleStringProperty("defaultUser");
            public StringProperty usernameProperty() { return username; }
        }
        User userModel = new User();

        Label modelValueLabel = new Label();
        modelValueLabel.textProperty().bind(Bindings.concat("Model Username: ",
userModel.usernameProperty()));

        Label uiFieldLabel = new Label("UI Username Field:");
        TextField uiUsernameField = new TextField();

        // Bind the UI text field to the model's username property
        uiUsernameField.textProperty().bindBidirectional(userModel.usernameProperty());
        // Typing in the text field now directly updates the 'userModel' object,
        // and the 'modelValueLabel' updates automatically because it is bound to the model.

        root.getChildren().addAll(label1, textField1, label2, textField2, modelLabel,
modelValueLabel, uiFieldLabel, uiUsernameField);

        Scene scene = new Scene(root, 400, 400);
        primaryStage.setTitle("Bidirectional Binding Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
}
```

- The first part of the demo shows the simplest case: two UI controls (TextField) are kept perfectly in sync.
- The second part shows the most important use case (Lab Practice-3 compatible): synchronizing a UI control (uiUsernameField) with a data model property (userModel.usernameProperty()). This is the core of the Model-View-ViewModel (MVVM) pattern often used with JavaFX.

Introduction to FXML (AFTER ALL CODE-SIDE CONCEPTS)

Up to this point, we have built every single part of our UI in Java code. This gives us maximum control but has some drawbacks:

- UI structure is mixed with application logic.
- It's verbose. A simple form can take many lines of Java code.
- It's difficult for a non-programmer (like a UI/UX designer) to understand or modify the layout.

This is why FXML exists.

Why FXML? The "Separation of Concerns" Principle

FXML is a declarative, XML-based language for defining the structure of a JavaFX user interface. It allows you to separate the **View** (the UI layout, defined in FXML) from the **Controller** (the application logic, written in Java).

Benefits:

- **Separation of Concerns:** Your Java code is cleaner because it's not cluttered with UI construction code.
- **Readability:** The hierarchical nature of XML maps perfectly to the JavaFX Scene Graph, making the UI structure easy to visualize.
- **Designer-Friendly:** Designers can use visual tools like **Scene Builder** to create and modify FXML files without touching the Java code.
- **Rapid Prototyping:** Quickly mock up UIs in FXML to test layouts and ideas.

The Relationship: FXML, Controller, and Java Code

Here's how they work together:

1. **The FXML File (.fxml):** Defines *what* the UI contains (Buttons, Labels, Layouts) and their properties (text, color, layout constraints). It can also link UI elements to a Controller.
2. **The Controller Class (.java):** Contains the application logic. It defines *how* the UI behaves. It holds the event handler methods and can access the UI elements defined in the FXML.
3. **The Main Application Class (.java):** Loads the FXML file, which in turn creates the UI and links it to the Controller.

Creating a Simple FXML Application

Let's convert our simple Login Form into an FXML-based application.

Step 1: Create the FXML File (LoginForm.fxml)
This file defines the layout and controls.

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.PasswordField?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.GridPane?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.text.Text?>

<GridPane fx:controller="LoginFormController" xmlns:fx="http://javafx.com/fxml/1"
          alignment="CENTER" hgap="10" vgap="10" styleClass="root-pane">
    <padding>
        <Insets bottom="25" left="25" right="25" top="25"/>
    </padding>

    <Text text="Welcome" styleClass="welcome-text"
          GridPane.columnIndex="0" GridPane.rowIndex="0" GridPane.columnSpan="2"/>

    <Label text="User Name:"
           GridPane.columnIndex="0" GridPane.rowIndex="1"/>

    <TextField fx:id="userTextField"
               GridPane.columnIndex="1" GridPane.rowIndex="1"/>

    <Label text="Password:"
           GridPane.columnIndex="0" GridPane.rowIndex="2"/>

    <PasswordField fx:id="passwordField"
                   GridPane.columnIndex="1" GridPane.rowIndex="2"/>

    <HBox alignment="BOTTOM_RIGHT" spacing="10"
          GridPane.columnIndex="1" GridPane.rowIndex="4">
        <Button text="Sign In" onAction="#handleSignInButtonAction"/>
    </HBox>

    <Text fx:id="actionTarget"
          GridPane.columnIndex="1" GridPane.rowIndex="6"/>

</GridPane>
```

Key FXML Elements:

- <?import ...?>: Imports the required JavaFX classes.
- <GridPane fx:controller="LoginFormController" ...>: The root element. The fx:controller attribute links this FXML to its Controller class.
- <TextField fx:id="userTextField" ...>: The fx:id attribute gives this TextField a unique ID. This allows the Controller to access it.
- <Button ... onAction="#handleSignInButtonAction"/>: The onAction attribute links the button's ActionEvent to the handleSignInButtonAction method in the Controller. The # indicates it's a method reference.

Step 2: Create the Controller Class (LoginFormController.java)
This class contains the logic.

```
import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.scene.control.PasswordField;
import javafx.scene.control.TextField;
import javafx.scene.paint.Color;
import javafx.scene.text.Text;

public class LoginFormController {

    // --- Injected UI Elements ---
    // The @FXML annotation tells the FXMLLoader to inject the corresponding
    // UI element from the FXML file into these fields.
    // The field name MUST match the fx:id in the FXML.
    @FXML
    private TextField userTextField;

    @FXML
    private PasswordField passwordField;

    @FXML
    private Text actionTarget;

    // --- Event Handler Method ---
    // This method is linked from the FXML file's onAction attribute.
    @FXML
    protected void handleSignInButtonAction(ActionEvent event) {
        String username = userTextField.getText();
        String password = passwordField.getText();

        if ("admin".equals(username) && "secret".equals(password)) {
            actionTarget.setFill(Color.GREEN);
            actionTarget.setText("Sign in successful!");
        } else {
            actionTarget.setFill(Color.RED);
            actionTarget.setText("Sign in failed.");
        }
    }

    // Optional: An initialize method that is called after the FXML is loaded
    @FXML
    public void initialize() {
        System.out.println("LoginFormController has been initialized.");
        // You can set up initial states here, e.g., listeners
    }
}
```

Key Controller Elements:

- @FXML: This annotation is crucial. It marks fields and methods that should be linked to the FXML file.
- **Field Injection:** Fields like userTextField are automatically populated by the FXMLLoader with the corresponding UI controls from the FXML, as long as the fx:id matches the field name.
- **Event Handler Method:** The handleSignInButtonAction method is just a regular Java method. The @FXML annotation is good practice but not strictly required for handler methods if they are public.

Step 3: Create the Main Application Class (MainFxmlApp.java)
This class is responsible for loading the FXML.

```
import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class MainFxmlApp extends Application {

    @Override
    public void start(Stage primaryStage) throws Exception {
        // --- Load the FXML file ---
        // The FXMLLoader reads the FXML, creates the Scene Graph,
        // and wires it up to the controller.
        Parent root = FXMLLoader.load(getClass().getResource("LoginForm.fxml"));

        primaryStage.setTitle("FXML Welcome");

        Scene scene = new Scene(root, 350, 275);

        // Optional: Apply CSS to the FXML-loaded scene
        // scene.getStylesheets().add(getClass().getResource("login-styles.css").toExternalForm());

        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Key Main App Elements:

- FXMLLoader.load(...): This is the static method that does all the magic. It parses the FXML file and returns the root Node of the Scene Graph (in this case, the GridPane).
- The rest of the start method is the same as before: create a Scene, set it on the Stage, and show it.

What changes where and how?

- **UI Construction:** Moves from procedural Java code in the start() method to declarative XML in the .fxml file.
- **Event Handling Logic:** Moves from anonymous classes or lambdas inside the start() method to dedicated methods (e.g., handleSignInButtonAction) inside the Controller class.
- **Control References:** Instead of creating local variables for controls (e.g., Button btn = new Button()), you declare fields in the Controller with the @FXML annotation and an fx:id in the FXML.

Scene Builder: Visual FXML Design

While you can write FXML by hand, it's much easier and faster to use **JavaFX Scene Builder**.

- **What it is:** A standalone visual design tool that allows you to drag and drop UI components, arrange them in layouts, modify their properties, and link them to a controller, all without writing any XML. It generates the FXML file for you.
- **Workflow:**
    1. Design your UI in Scene Builder.
    2. In the "Controller" panel, specify the name of your controller class.
    3. For each control you need to access in code, assign an fx:id in the "Code" panel.
    4. For each event handler, specify the method name in the "On Action" (or similar) field in the "Code" panel.
    5. Save the FXML file.
    6. In your IDE, you can often "generate" the controller code stubs from the FXML, which creates the @FXML fields and methods for you to fill in.
    7. Write the logic in the generated controller methods.

Using Scene Builder significantly speeds up UI development and is the standard way of working with FXML.

UX Considerations, Accessibility, and Styling Best Practices

Building a functional UI is only half the battle. A good application must also be usable, accessible, and maintainable.

Principles of Good User Experience (UX) Design

- **Clarity:** The UI should be intuitive. Users should understand what to do without reading a manual. Use clear labels, consistent icons, and logical grouping of controls.
- **Consistency:** The application should be consistent within itself and with platform conventions. Buttons, menus, and shortcuts should behave as the user expects.
- **Feedback:** Provide immediate and clear feedback for user actions. When a user clicks a button, show that it was pressed. If an operation takes time, use a ProgressBar or an indeterminate indicator. Use status messages for success or failure.
- **Efficiency:** Design layouts that minimize clicks and streamline common workflows. Use sensible defaults and remember user choices.
- **Simplicity:** Avoid clutter. A clean, minimalist design is often more effective than a screen filled with too many options. Hide advanced features until they are needed.

Basic Accessibility Considerations

Accessibility (a11y) means designing your application to be usable by people with disabilities, such as visual or motor impairments.

- **Keyboard Navigation:** Ensure all interactive controls can be reached and operated using the Tab key and arrow keys. JavaFX's default controls handle this well, but be mindful when creating custom components.
- **Screen Readers:** Provide meaningful text for controls. For graphical buttons, use a Tooltip to provide a textual description.
- **Color Contrast:** Ensure that text has sufficient contrast against its background to be readable by people with low vision. Use tools to check your color palettes.
- **Scalability:** Use scalable layouts (GridPane, BorderPane) and resizable fonts so users can adjust the application's size to their needs.

Styling Best Practices (Reiteration)

- **Use External CSS:** Always prefer external .css files over inline styles. It makes your application easier to theme, maintain, and debug.
- **Use a Naming Convention:** Use a consistent naming scheme for your style classes (e.g., BEM - Block, Element, Modifier, like .user-profile__button--danger).
- **Create a Base Stylesheet:** Define common styles (colors, fonts, padding) in a base stylesheet that is used across your entire application to ensure a consistent look and feel.
- **Leverage Pseudo-classes:** Make your UI feel responsive by providing visual feedback for :hover, :pressed, and :focused states on all interactive elements.

Formative Quiz (Sample Questions)

Short Multiple Choice Questions (MCQ):

1. Which layout pane arranges its children in a single vertical column?
   a) HBox
   b) GridPane
   c) VBox
   d) BorderPane
2. What is the primary purpose of FXML?
   a) To perform complex background calculations.
   b) To separate the user interface layout from the application logic.
   c) To define animations and transitions.
   d) To connect directly to a database.
3. To make two TextFields automatically sync their text with each other, you should use:
   a) An ActionEvent listener.
   b) bind() (unidirectional binding).
   c) bindBidirectional().
   d) A Timeline.
4. In the JavaFX event propagation model, what is the first phase called?
   a) Bubbling Phase
   b) Handling Phase
   c) Filtering/Capturing Phase
   d) Consumption Phase

Short Code Snippet Question:

You have a Button named submitButton and a CheckBox named termsCheckBox. Write a single line of JavaFX code using **property binding** to ensure the submitButton is disabled until the termsCheckBox is selected.

Answer:
submitButton.disableProperty().bind(termsCheckBox.selectedProperty().not());
(Or Bindings.not(termsCheckBox.selectedProperty()))

Reading Assignment

- **Official Oracle JavaFX Documentation:**
  - Getting Started with JavaFX: https://openjfx.io/openjfx-docs/
  - Working with JavaFX UI Components
  - JavaFX CSS Reference Guide
- **Recommended Textbooks:**
  - *Pro JavaFX 9* by Johan Vos, Weiqi Gao, Stephen Chin, and Dean Iverson (or a newer edition if available). Focus on chapters covering Layouts, UI Controls, Event Handling, and Properties/Binding.